



Rotary Positional Encodings for Graph Transformers



Cederik Levi Höfs

Supervisors: Petar Veličković
Isaac Reid

Department of Engineering
University of Cambridge

This dissertation is submitted for the degree of
Master of Philosophy in Machine Learning and Machine Intelligence

Darwin College

14th August 2025

Abstract

Rotary Positional Encodings (RoPE) have proven highly effective for transformers in natural language processing. Applying transformers to graph-structured data can help model long-range interactions, thereby improving performance on various tasks. This thesis investigates the adaptation of RoPE to graph transformers as a novel positional encoding modality.

By using spectral features derived from the graph Laplacian, RoPE can inform the otherwise ignorant attention mechanism of the graph’s structure. A theoretical analysis shows that beneficial properties of RoPE are preserved while also revealing a failure case – graphs without node features.

The approach is implemented within the GraphGPS framework for a systematic evaluation of design choices. Empirical results on diverse datasets demonstrate that the method is effective, albeit not on par with concatenated Laplacian Positional Encodings. Hypotheses about the method’s failure mode are formulated and then tested through synthetic experiments.

While this work advances the understanding of applying RoPE to graph transformers, several research questions remain. Addressing these questions is crucial for unlocking the full potential of this approach and advancing the field of graph machine learning.

Declaration

I, Cederik Levi Höfs of Darwin College, being a candidate for the MPhil in Machine Learning and Machine Intelligence, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.

Cederik Levi Höfs
14th August 2025

Declaration

Software Declaration

The following software was used in the preparation of this report without modification:

- **Python 3, NumPy and SciPy:** Used for all experimental implementations, data processing, and analysis.
- **PyTorch Geometric:** Used for deep learning model implementations and training.
- **Matplotlib:** Used for generating figures and visualisations throughout the report.
- **LaTeX:** Used for typesetting this report using the Cambridge University PhD thesis template.

The **GraphGPS** repository¹ formed the basis of implementations and experimental setups. An optional sparse eigendecomposition and spectral caching system was added. Multi-head attention using multidimensional RoPE on the output of configurable encoders was implemented. Several existing benchmark datasets as well as custom synthetic ones were included for use in the experiments.

All experimental code, model implementations, and analysis scripts were developed specifically for this project. The software listed above was used in **Chapter 4** for the implementation and evaluation of the proposed method.

Word Count: Approximately 12,600 words (excluding declarations, bibliography, photographs and diagrams, but including tables, footnotes, figure captions and appendices).

Cederik Levi Höfs
14th August 2025

¹<https://github.com/rampasek/graphgps>

Declaration

Prior Work

Minor parts of the work presented in this thesis have been submitted in form of a report on a joint mini-project in the course *L65: Geometric Deep Learning*. This joint initial exploration of using RoPE on graphs with Leonhard Vulpius sparked the motivation for this work. Relevant bits/ideas that were reused for reference are:

- The formulation of RoPE on graphs (cf. unified notation in [Section 3.1](#))
- A limited, tentative implementation (reworked and extended as documented in [Section 3.3](#)).
- Recovering RoPE and STRING for suitable graphs (corrected mistakes regarding DeepSet encoding, cf. [Section B.1](#)).
- The possibility of interpreting RoPE as a Fourier series (cf. [Section 3.2.3](#)).
- Small-scale experiments on the `Peptides-functional` dataset (now obsolete as rerun after changing the implementation).

Cederik Levi Höfs
14th August 2025

I dedicate this thesis to my loving parents
and all friends I have met along the way.

Acknowledgements

I would like to thank my supervisors, Isaac Reid and Petar Veličković, for their support and guidance. Thanks also go to Leonhard Vulpius, as this project emerged from our collaboration. Grateful acknowledgement is also made to Studienstiftung des deutschen Volkes and the German Academic Exchange Service, without whose financial support my time in Cambridge would not have been possible. Further financial support, for which I am very grateful, was provided by the Dr. Jürgen und Irma Ulderup Stiftung.

Table of contents

List of figures	xix
List of tables	xxi
1 Introduction	1
1.1 Graph Machine Learning	1
1.2 Research Objectives	1
1.3 Contributions	2
1.4 Thesis Organisation	3
1.5 Summary	3
2 Background and Related Work	5
2.1 Graphs and Transformers	6
2.1.1 Graph Convolutional Networks (GCN)	7
2.1.2 Transformer	8
2.1.3 GraphGPS	10
2.2 Rotary Positional Encoding	11
2.2.1 Multidimensional RoPE / STRING	12
2.2.2 Theoretical Properties	13
2.3 Graph Positional Encoding Approaches	14
2.3.1 Spectral Graph Theory	14
2.3.2 Laplacian Positional Encoding	16
2.4 Graph Machine Learning Benchmarks	18
2.4.1 Long-Range Graph Benchmark	18
2.4.2 Open Graph Benchmark	20
2.4.3 Stochastic Block Models	20
2.4.4 Subreddits	21
2.4.5 Single-Graph Datasets	21

2.4.6	Heterophilic Graphs	21
2.4.7	Motif Detection	22
2.5	Benchmarking Positional Encodings	22
2.6	Summary	23
3	Rotary Positional Encoding for Graphs	25
3.1	Definition	25
3.2	Theoretical Analysis	26
3.2.1	Invariance	26
3.2.2	Equivariance	27
3.2.3	Interpretations	27
3.2.4	Graph Transformers without Semantic Features	28
3.2.5	Recovering RoPE/STRING	29
3.2.6	Expressivity	30
3.2.7	Computational Complexity	31
3.2.8	Design Choices	31
3.3	Implementation	32
3.3.1	Spectral Cache	33
3.3.2	Model Nomenclature	33
3.4	Summary	34
4	Experiments	35
4.1	Overview	35
4.2	Unit Tests	36
4.3	Molecular Tasks (Peptides & molpcba)	39
4.3.1	Non-Long-Range Tasks	41
4.3.2	Conclusion	42
4.4	Structural Tasks (PATTERN & REDDIT-MULTI-5K)	42
4.4.1	Encoder Design	44
4.4.2	Shared Omega Matrices	44
4.4.3	Expressivity Breakdown for GT+RoPE	45
4.5	Single-Graph Datasets	45
4.5.1	Correlation of Effects of LapPE and RoPE	46
4.5.2	Hyperparameter Tuning	48
4.6	Heterophilic Graphs (Minesweeper)	48
4.6.1	Lanczos' Algorithm	49
4.7	Motif Detection (Cycles & Caveman)	50

4.7.1	Hyperparameter Sweeps	50
4.8	Coloured Subcomponent Diameter	51
4.8.1	Rewiring Robustness	51
4.8.2	Sign Flip Invariance	51
4.9	Grid Graphs (CIFAR-10)	53
4.9.1	Recovering ViT	53
4.10	Summary	54
5	Conclusion and Future Directions	57
5.1	Impact	57
5.2	Strengths and Limitations	58
5.3	Open Questions and Future Work	58
5.4	Summary	60
	References	61
	Appendix A Notation	69
	Appendix B Proofs	71
B.1	Recovering RoPE	71

List of figures

2.1	Overview of RoPE and LapPE	5
2.2	Schematic of a GPS layer	11
2.3	Eigendecomposition of the Path Graph	16
2.4	Graph Motifs for Detection Task	22
3.1	Multi-dimensional RoPE	26
4.1	RoPE Attention Modulation	37
4.2	Effective Rank Evolution of Ω	38
4.3	Peptides-struct Performance vs k	40
4.4	molpcba Performance vs k	42
4.5	GT Performance on PATTERN	43
4.6	Encoder Depth on PATTERN	44
4.7	Ω -Design Comparison on PATTERN	45
4.8	LapPE and RoPE Effect Correlation	47
4.9	Performance on Minesweeper	49
4.10	Lanczos' Algorithm on Minesweeper Grid	50
4.11	Performance on the Caveman Task	52
4.12	Rewired Grid Graph	53
4.13	Model Comparison on CIFAR-10	55

List of tables

2.1	Dataset Summary	19
4.1	Unit Test Results	36
4.2	Peptides-func Performance	39
4.3	Single-Graph Dataset Performance	46

Chapter 1

Introduction

This introductory chapter provides an overview of graph machine learning, outlines the research objectives and contributions of this thesis and presents the organisational structure of the document.

1.1 Graph Machine Learning

Many real-world systems, such as molecules, social networks and biological interactions, can be naturally represented as graphs. Machine learning is the task of identifying patterns in data through the numerical optimisation of statistical models. Advances in computational power and data availability have led to breakthroughs in fields like computer vision, speech recognition and natural language processing, enabling countless downstream applications.

1.2 Research Objectives

Much of the recent success in machine learning has been driven by the transformer architecture, a task-agnostic approach applicable to a wide range of domains. Applying transformers to graph-structured data is a promising method for modelling long-range dependencies, which are present in many real-world tasks.

In many areas of machine learning, the current trend is to scale towards larger models with fewer hard-coded inductive biases ([Wang et al., 2024](#)). Graphs can model more irregular structures than for example image or language data. Still, future developments are expected to move towards large pre-training datasets in the context of graph machine learning ([Liu et al., 2025](#)). Therefore, exploring approaches that have

proven effective in Large Language Models (LLMs) and applying them to graphs is of great interest. Improving the performance of such models has implications for numerous important applications, including drug discovery (Gao et al., 2024), recommendation systems (Min et al., 2022b) or hate speech detection (Hebert et al., 2022).

In the training of LLMs, Rotary Positional Encoding (RoPE) (Su et al., 2023) has become the standard method for making such models aware of the order of tokens. Similarly, Laplacian Positional Encoding (LapPE) (Kreuzer et al., 2021) is a technique for informing machine learning models about the structure of graphs.

This thesis is concerned with exploring the previously unstudied application of RoPE on spectral features in graph transformers. The following research questions are investigated:

- What theoretical properties does RoPE on graphs have?
- How does RoPE perform on real-world datasets?
- Are there differences between purely semantic or structural tasks as special cases?

1.3 Contributions

This thesis makes the following contributions:

- RoPE for use on graphs is introduced (Section 3.1) and implemented on top of the GraphGPS framework by Rampásek et al. (2022) (Section 3.3).
- A theoretical analysis of some useful properties of RoPE on graphs is carried out in Section 3.2. Recovering conventional RoPE for path and grid graphs is desirable and possible (Section 3.2.5).
- Evaluations on real-world datasets show that the method is effective, but not on par with the state-of-the-art (Section 4.3).
- The design space of possible architectural choices like the number of eigenvectors used in the encoding is explored on diverse datasets (Section 4.4).
- Synthetic experiments are performed to answer fundamental questions about the workings of RoPE on graphs (Sections 4.8 and 4.9).

1.4 Thesis Organisation

The remainder of this thesis is organised as follows: [Chapter 2](#) reviews the key foundational concepts used in this work, including architectures, positional encodings and benchmark datasets. Building on this, [Chapter 3](#) details the proposed new methodology, offering theoretical insights and remarks on the implementation. Subsequently, [Chapter 4](#) provides a systematic empirical validation of the approach through extensive experiments on both real-world and synthetic datasets. Finally, [Chapter 5](#) consolidates the main findings, discusses the limitations of this work and proposes promising avenues for future research. A summary of the mathematical notation used throughout this thesis which may at times be non-standard is provided in [Chapter A](#). The appendix also contains a somewhat technical proof ([Chapter B](#)).

1.5 Summary

Having established the broad motivation and goals of this work, the main objective of this thesis is clear: Investigating the use of RoPE on graphs.

The following chapter provides the necessary technical background on graph transformers and positional encodings, laying the groundwork for the contributions presented in the following chapters.

Chapter 2

Background and Related Work

This chapter reviews the fundamental knowledge that found application in this thesis. It begins by introducing graph neural networks and the transformer architecture, which are combined in the GraphGPS framework. We then discuss two key positional encoding methods that originated from the same original positional encoding in transformers: Rotary Positional Encoding in LLMs and Laplacian Positional Encoding for Graph Neural Networks (GNNs) (cf. [Figure 2.1](#)). The synthesis of these two techniques is the central focus of this work. Finally, we introduce the benchmark datasets used to evaluate the proposed approach and prior results thereon.

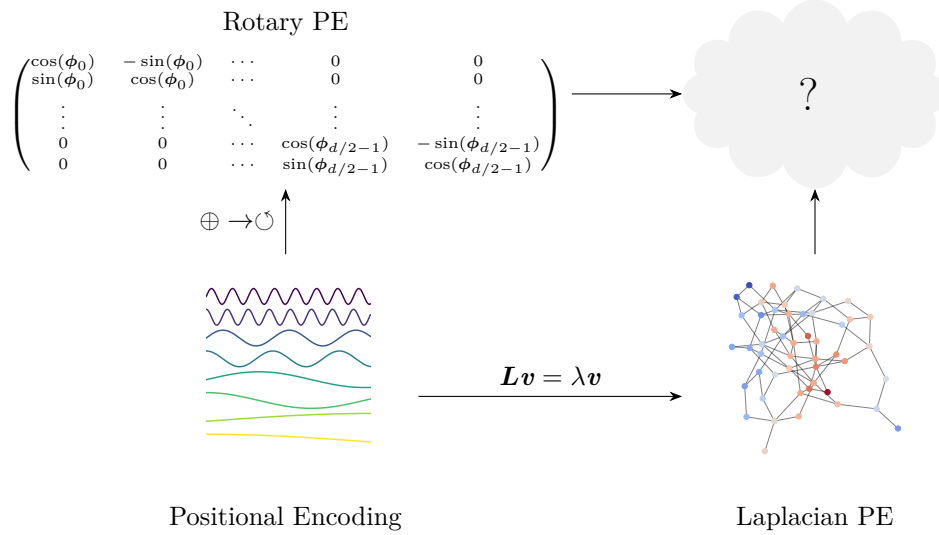


Fig. 2.1 The two main ideas combined in this work: Rotary Positional Encoding substituting addition with rotation and Laplacian Positional Encodings for graphs which are based on eigendecompositions.

2.1 Graphs and Transformers

Recall that a graph is a set of nodes V connected in pairs by edges $E \subseteq V \times V$. All graphs considered in this work are undirected and simple, meaning that they have no self-loops and no multiple edges.

In supervised graph machine learning, the general task is to learn a parameterised function $f_{\theta} : \mathcal{G} \rightarrow \mathcal{Y}$, where $\mathcal{G}$ is a set of graphs and $\mathcal{Y}$ is a set of labels. For classification tasks, $\mathcal{Y}$ is typically the C -dimensional probability simplex Δ^C , while for (multidimensional) regression tasks, it is $\mathbb{R}^C$. The nodes and edges of the input graph may be labelled with features, though in this work, only node features are considered, denoted by the matrix $\mathbf{X} \in \mathbb{R}^{N \times d_{in}}$.

The models in this thesis compute a hidden representation of the input graph. This is then used to predict either logits for classification or a continuous value for regression. The model parameters θ are learned by minimizing a loss function $\mathcal{L}$ over a training set $\mathcal{D}_{\text{train}} \subset \mathcal{G} \times \mathcal{Y}$:

$$\theta^* = \operatorname{argmin}_{\theta} \mathcal{L}(\theta; \mathcal{D}_{\text{train}})$$

The corresponding loss functions are the cross-entropy loss for classification and the mean absolute error for regression:

$$\begin{aligned} \mathcal{L}_{\text{CE}}(\theta; \mathcal{D}) &= -\frac{1}{|\mathcal{D}|} \sum_{(G,y) \in \mathcal{D}} \sum_{c \in [C]} y_c \log f_{\theta}(G)_c, \\ \mathcal{L}_{\text{MAE}}(\theta; \mathcal{D}) &= \frac{1}{|\mathcal{D}|} \sum_{(G,y) \in \mathcal{D}} \sum_{c \in [C]} |y_c - f_{\theta}(G)_c|. \end{aligned}$$

The learned function should naturally be adapted to the fact that the input is a graph. Graphs cannot be canonically ordered, so neural networks which typically operate on vectors have to be modified to account for this. Further, the nodes are endowed with more than just set structure – the edges of the graph.

The adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ captures this structure:

$$\mathbf{A}_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

Consider a permutation matrix $\mathbf{P} \in \{0, 1\}^{N \times N}$ that reorders the nodes of a graph. A function f is *permutation invariant* if

$$f(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^{\top}) = f(\mathbf{X}, \mathbf{A})$$

and *permutation equivariant* if

$$f(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^\top) = \mathbf{P}f(\mathbf{X}, \mathbf{A})$$

for any permutation matrix $\mathbf{P}$. Graph-level prediction tasks require invariance to node permutations, while node-level tasks require equivariance (Bronstein et al., 2021).

Fortunately, the question of how to build neural network architectures for graphs has been well studied (Hamilton, 2020):

2.1.1 Graph Convolutional Networks (GCN)

One of the first and simplest graph neural networks to be proposed was the Graph Convolutional Network (GCN) by Kipf and Welling (2016). Self-loops are added to the adjacency matrix $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$. A single GCN layer then propagates features according to

$$\mathbf{X}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{X}^{(l)} \mathbf{W}^{(l)}\right), \quad \mathbf{X}^{(0)} = \mathbf{X},$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{d \times d}$ are trainable weight matrices and σ is an element-wise nonlinearity for which we use $\text{ReLU}(x) = \max(0, x)$. The symmetrically normalised adjacency matrix $\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ allows for information flow along the edges of the graph. The overall computational cost for each GCN layer is $\mathcal{O}(|E|d + Nd^2)$ as the matrix products with $\mathbf{A}$ are between sparse and dense matrices (Kipf and Welling, 2016) which makes this a scalable method for graphs where $|E| \sim N$. Stacking L layers yields node embeddings $\mathbf{X}^{(L)}$ that integrate increasingly wider context (the L -hop neighbourhood). This offers a principled way to treat graphs using neural networks, hard-wiring inductive biases such as equivariance and locality into the model.

On the other hand, these properties entail several problems. Rampásek et al. (2022) summarise the vast literature on key issues pertaining to GNNs: Oono and Suzuki (2019) show that GNNs invariably suffer from oversmoothing. Only transmitting information along edges naturally constitutes a bottleneck as observed by Alon and Yahav (2020).

While many more general graph neural network architectures have been proposed, this discussion is limited to this original design as this forms the basis of all GNN applications in this work. GCN is selected for its simplicity, which allows for a clearer analysis of the marginal impact of the novel positional encoding scheme under

investigation, without confounding factors from a more complex message-passing backbone.

Only the direct comparison with GraphGPS on one dataset involved a different message-passing neural network, namely GatedGCN (Bresson and Laurent, 2017). This architecture incorporates edge features that rescale the messages being passed but is otherwise similar to GCN in its formulation.

2.1.2 Transformer

Multi-head Attention

Meanwhile, in a different area of machine learning, language modelling has been revolutionised by the transformer architecture (Vaswani et al., 2017). A standard multi-head self-attention block can be seen as a soft, neural lookup modelling pairwise interactions between tokens (used interchangeably with nodes in this work).

Given node features $\mathbf{X} \in \mathbb{R}^{N \times d}$, the module computes queries, keys, and values via affine projections with weight matrices $\mathbf{W}_{Q,K,V} \in \mathbb{R}^{d \times d}$. These projections are then split into H heads, each of dimension $d_H = d/H$. The core of the attention mechanism are pairwise similarity scores κ between nodes which are normalised to take a convex combination of the value vectors of the nodes:

$$\begin{aligned} \mathbf{Q}, \mathbf{K}, \mathbf{V} &= \mathbf{X} \mathbf{W}_{Q,K,V} \\ \mathbf{h}_i &= \mathbf{A}_i \mathbf{V}_i \\ &= \text{softmax}(\kappa_i) \mathbf{V}_i \\ &= \text{softmax} \left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{d_H}} \right) \mathbf{V}_i \end{aligned} \tag{2.1}$$

where $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i$ are the query, key, and value matrices for the i -th head. The outputs from all heads are concatenated and passed through a final linear projection $\mathbf{W}_O \in \mathbb{R}^{d \times d}$:

$$\text{Attention}(\mathbf{X}) = [h_1(\mathbf{X}), \dots, h_H(\mathbf{X})]_2 \mathbf{W}_O.$$

For brevity, we omit the multi-headed nature of the attention mechanism and bias terms in the following.

Performer

Instantiating the full attention matrix (Equation (2.1)) is computationally expensive, as it has size N^2 . The Performer architecture by Choromanski et al. (2020) is a linear-time alternative to the above mechanism. Dubbed Fast Attention Via Orthogonal Random features, FAVOR+, this approach offers an unbiased approximation to softmax attention.

Transformer Block

A full transformer block as used in LLMs consists of a multi-head attention layer followed by a feed-forward network (a Multi-Layer Perceptron (MLP)). In practice, additions like layer normalisation (Ba et al., 2016) and residual connections (He et al., 2016) are applied before and after the MLP.

$$\begin{aligned}\text{MLP}(\mathbf{X}) &= \text{ReLU}(\mathbf{X}\mathbf{W}_1)\mathbf{W}_2 \\ \text{Transformer}(\mathbf{X}) &= \text{MLP}(\text{Attention}(\mathbf{X}))\end{aligned}$$

Positional Encoding

As currently stated, the transformer architecture is fully equivariant to token permutations. This is not desirable for text data, as the order of tokens usually carries important information. Originally it was common to add sinusoids to the input features with frequencies decaying exponentially from a base frequency of $\omega_0 = 10000$ to supply the model with positional information (Vaswani et al., 2017):

$$\begin{aligned}\mathbf{T}_{i,2j} &= \sin\left(i\omega_0^{-\frac{j}{d/2}}\right) & \mathbf{T}_{i,2j+1} &= \cos\left(i\omega_0^{-\frac{j}{d/2}}\right) \\ \mathbf{X}_{\text{PE}} &= \mathbf{X} + \mathbf{T}\end{aligned}\tag{2.2}$$

Modalities

While celebrating great successes with models pre-trained on large corpora of text (Brown et al., 2020; Comanici et al., 2025; Grattafiori et al., 2024), transformers are also applied in other domains like

- Computer Vision (Dosovitskiy et al., 2020; Liu et al., 2021)
- Speech Recognition (Radford et al., 2023)

- Image Generation / Diffusion (Peebles and Xie, 2023)

with positional encodings being part of all the above models.

Naively applying pure transformers to graphs would consider the node features as a multiset of tokens (or equivalently, perform computation along a fully connected graph). While this is a valid approach and ensures correct behaviour under permutations, such models would be blind to the graph structure itself.

Instead, attention can offer benefits when combined with message passing.

2.1.3 GraphGPS

Graph Transformers

To alleviate the aforementioned shortcomings of GNNs, many transformer-based graph machine learning models have been proposed. Min et al. (2022a) compiled a comprehensive summary of the field; the interested reader is referred to this work. Veličković et al. (2018) pioneered the use of local attention during message passing. A global attention mechanism has been part of various proposals (Dwivedi and Bresson, 2020).

Ying et al. (2021)’s Graphormer encodes structural information like node centrality and shortest path distances by directly adding biases to the attention score matrix. This forces its instantiation and is hence not compatible with the use of linear attention mechanisms. GraphiT (Mialon et al., 2021) is a method that explicitly multiplies a kernel matrix with the exponentiated attention scores.

Structural information derived from the eigenvectors of the graph Laplacian found use in SAN (Kreuzer et al., 2021).

GraphGPS

The GraphGPS framework by Rampášek et al. (2022) generalised many of these proposals by combining local message-passing and global attention into the framework of the GPS layer. Its modular design allows for easy experimentation with new architectures. The GraphGym-based (You et al., 2020) training infrastructure allows for focus on the design of the model and subsequent configuration of hyperparameters.

The GPS layer applies the composite mapping (note that we omit edge features as we do not make use of them in any of our experiments)

$$\text{GPS}(\mathbf{X}, \mathbf{A}) = \text{MLP}(\text{MPNN}_{\mathbf{A}}(\mathbf{X}) + \text{Attention}(\mathbf{X})). \quad (2.3)$$

The operator MPNN can be any message-passing neural network aggregating information along edges. Attention is a transformer-style attention module acting on all nodes simultaneously allowing for global information exchange. Both submodules are bypassed by residual connections and summed together, followed by an MLP (again with a skip connection).

The complete models used are L repeated blocks of the module shown in [Figure 2.2](#). Each submodule (GCN, attention, MLP) has dropouts of configurable probability enabled. Batch normalisation ([Ioffe and Szegedy, 2015](#)) is also applied in each of these submodules.

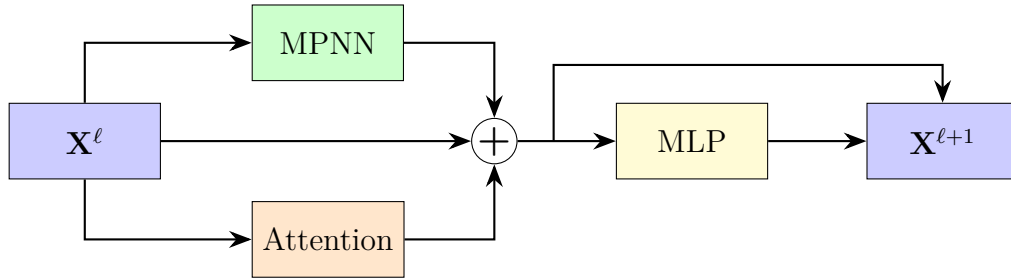


Fig. 2.2 The GPS layer combines message-passing and attention modules (with residual connections) by summing, followed by an MLP. This hybrid layout subsumes many previous graph transformer designs.

In this work, GCN was used as the message-passing neural network and either vanilla multi-head attention or Performer as the attention module.

Similar as in the language modelling setting, the usefulness of permutation-equivariant vanilla transformer layers is limited as it is unaware of the edge structure of the graph (in contrast to GNNs). GraphGPS therefore implements and offers multiple structural and positional encoding methods to inform the model of the graph structure. These will be discussed after a short detour into modern positional encoding in language modelling.

2.2 Rotary Positional Encoding

We have seen above how sinusoids have been used to inform transformer models of the order of tokens in a sequence. In LLMs, this has recently been superseded by a more elegant approach: RoPE by [Su et al. \(2023\)](#).

Instead of concatenating or adding positional encodings to the input and computing vanilla attention of the result, the attention mechanism is modified, modulating the attention score (pairwise scalar products) and thereby infusing positional information

at every layer of the model. RoPE achieves this by rotating the key and query vectors in two-dimensional subspaces with varying frequencies based on the position of the token in the sequence. After this, attention is computed as usual.

This modality processes the positional information in a separate channel from the semantic information while still allowing for their interaction.

To write this succinctly, we use the operator $\mathcal{R}$ stacking block-diagonal rotation matrices defined in:

$$\mathcal{R} : \mathbb{R}^{\frac{d}{2}} \rightarrow \mathbb{R}^{d \times d}, \quad \mathcal{R}(\phi) = \text{diag} \left(\left(\begin{array}{cc} \cos(\phi_l) & -\sin(\phi_l) \\ \sin(\phi_l) & \cos(\phi_l) \end{array} \right)_{l \in [d/2]} \right) \quad (2.4)$$

The rotary frequencies $\omega \in \mathbb{R}^{d/2}$ used to derive the angles $\phi_i = \omega i$ for each chunk may be learned or be fixed and decay exponentially, analogous to [Equation \(2.2\)](#):

$$\omega_l = \omega_0^{-\frac{l}{d/2}}$$

One-dimensional RoPE is then given by:

$$\{\mathbf{q}, \mathbf{k}\}_i^{\text{RoPE}} = \mathcal{R}(\omega i) \{\mathbf{q}, \mathbf{k}\}_i$$

For use in Vision-Language Models (VLMs), more advanced frequency allocation methods have been discussed ([Hua et al., 2024](#); [Li et al., 2025](#)). In this work, frequencies are generally kept learnable as the positional feature space is not as clearly interpretable regardless.

2.2.1 Multidimensional RoPE / STRING

RoPE has been generalised to more than one dimension, e.g. for use in Vision Transformers (ViT) ([Heo et al., 2024](#)).

An original approach by [Ostmeier et al. \(2024\)](#) using learned higher-dimensional rotations was followed by the more efficient STRING ([Schenck et al., 2025](#)). Their method naturally gives translational invariance and forms the basis of this work.

Successive applications of two-dimensional rotations are equivalent to taking a linear combination of rotation angles which are themselves linear combinations of the positional features $\mathbf{t}_i \in \mathbb{R}^k$. These can e.g. be the x and y coordinates of the patches of an image. Writing this as matrix vector product followed by the rotation operator gives a more concise formulation than that of repeated applications of rotations:

$$\{\mathbf{q}, \mathbf{k}\}_i = \mathcal{R}(\mathbf{\Omega}^\top \mathbf{t}_i) \{\mathbf{q}, \mathbf{k}\}_i \quad (2.5)$$

In this case, $\mathbf{\Omega} \in \mathbb{R}^{k \times d/2}$ maps the k positional features to the $d/2$ angles. Using this notation with $\mathcal{R}$ makes clear how the columns of $\mathbf{\Omega}$ (each corresponding to one angle) "gauge" the $\mathbf{t}$ -feature along different directions with their scalar product resembling the k -space encountered in physics.

As $\mathbf{X}$ is already in use for the node features, the variable pair $\mathbf{\Omega}, \mathbf{T}$ is used instead to uphold this resemblance. Explicitly denoting the final linear transformation $\mathbf{\Omega}$ instead of absorbing it into the encoder makes clear that we expect the dimension of the positional feature space k to be significantly smaller than the number of angles $d/2$.

The relevant positional feature space must not necessarily be dimensionally homogeneous, one can e.g. incorporate temporal differences (Assran et al., 2025).

2.2.2 Theoretical Properties

Invariance

Taking the scalar product of query and key vectors to obtain the attention score matrix κ effectively inverts one of their rotations. This makes the modulation of the attention mechanism purely a function of the pairwise difference of the positional features:

$$\kappa_{ij}^{\text{RoPE}} = (\mathcal{R}(\phi_i) \mathbf{q}_i)^\top \mathcal{R}(\phi_j) \mathbf{k}_j = \mathbf{q}_i^\top \mathcal{R}(\phi_j - \phi_i) \mathbf{k}_j = \mathbf{q}_i^\top \mathcal{R}(\mathbf{\Omega}^\top (\mathbf{t}_j - \mathbf{t}_i)) \mathbf{k}_j \quad (2.6)$$

For text and image data, this shift invariance has a clear interpretation: Only the relative distance of tokens matters as opposed to their absolute position.

Computational Efficiency

RoPE can be applied in linear time $\Theta(Nd)$ as the rotation matrix is block-diagonal and applied per-node. As noted by Su et al. (2023), it can therefore be applied in a very efficient manner: The even and odd parts of the query and key vectors are multiplied element-wise with $\sin(\phi_i)$ and $\cos(\phi_i)$ respectively and summed with appropriate signs.

Length Extrapolation

Contrary to initial assumptions by [Su et al. \(2023\)](#), RoPE does not decay with growing distance between tokens – instead, while analysing LLMs, [Barbero et al. \(2024\)](#) associate higher frequencies with positional attention heads and lower frequencies with semantic attention heads.

There are some hints that this approach allows for better generalisation over unseen lengths of sequences ([Press et al., 2021](#)), although this has been challenged by [Kazemnejad et al. \(2023\)](#). Simple rescaling of positions may in addition be helpful to achieve long context lengths ([Chen et al., 2023](#)).

2.3 Graph Positional Encoding Approaches

A second development was motivated by [Vaswani et al. \(2017\)](#)’s sinusoidal positional encodings: The resonance modes of a graph reveal a lot about its structure. This perspective can be seen as a discrete analogue of [Kac \(1966\)](#)’s famous question "Can you hear the shape of a drum?".

The eigenvectors of the graph Laplacian can thus be used to provide graph transformers with soft structural inductive bias for treating graphs ([Kreuzer et al., 2021](#)). By considering the sequence of tokens in NLP as a path graph we see that this approach generalises the positional encoding of [Vaswani et al. \(2017\)](#) while respecting properties we care about for graph data.

2.3.1 Spectral Graph Theory

The theoretical underpinning for this is provided by spectral graph theory, the study of graph structures through the lens of linear algebra. This section establishes the key concepts that are relevant to this work, closely following [Hamilton \(2020\)](#).

The degree matrix $\mathbf{D} = \text{diag} \left(\left(\sum_{j=1}^N A_{ij} \right)_{i \in [N]} \right)$ contains the degree of each vertex.

From this and the adjacency matrix one can define different variants of the graph Laplacian, the discrete analogue of the Laplace operator for functions on the graph:

- **Unnormalised:** $\mathbf{L} = \mathbf{D} - \mathbf{A}$
- **Symmetrically normalised:** $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$

The unnormalised Laplacian is the direct analogue of the continuous Laplacian differential operator, but its spectrum can be heavily influenced by nodes with high

degrees. The symmetric normalised Laplacian, $\mathbf{L}_{\text{sym}}$, has eigenvalues bounded in $[0, 2]$ (Chung, 1997). This may offer numerical stability benefits (Hamilton, 2020).

As these matrices are symmetric and real, their eigendecompositions are given by:

$$\mathbf{L}\psi_k = \lambda_k\psi_k \quad (2.7)$$

where $\lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_{N-1} \in \mathbb{R}$ are the eigenvalues and $\psi_0, \psi_1, \dots, \psi_{N-1} \in \mathbb{R}^N$ are the corresponding orthonormal eigenvectors (Horn and Johnson, 1985).

In matrix notation this diagonalisation can be written as:

$$\mathbf{L} = \mathbf{\Psi}\mathbf{\Lambda}\mathbf{\Psi}^T$$

where $\mathbf{\Psi} = [\psi_0, \psi_1, \dots, \psi_{N-1}]_2 \in \mathbb{R}^{N \times N}$ is the matrix of eigenvectors and $\mathbf{\Lambda} = \text{diag}(\lambda_0, \lambda_1, \dots, \lambda_{N-1}) \in \mathbb{R}^{N \times N}$ is the diagonal matrix of eigenvalues.

The eigenvalues and eigenvectors of the graph Laplacian encode fundamental structural properties of the graph. The smallest eigenvalue $\lambda_0 = 0$ corresponds to the trivial constant eigenvector $\mathbf{1}_N$ and the number of zero eigenvalues equals the number of connected components (Hamilton, 2020). The first non-zero eigenvalue, known as the algebraic connectivity or Fiedler (1973) value, measures how well-connected the graph is. In general, higher order eigenvectors resolve more fine-grained structural patterns of a graph.

Degeneracy

As is clear from Equation (2.7), eigenvectors are only defined up to a sign flip. An eigenvalue is called degenerate if it has a multiplicity greater than one, meaning that there are multiple linearly independent eigenvectors corresponding to it spanning eigenspaces $\mathbf{E}_i$. More generally, for degenerate eigenvalues, any linear combination of the corresponding eigenvectors is also an eigenvector for the same eigenvalue (Chung, 1997). Eigenvectors therefore come in sets of bases paired with their corresponding eigenvalue. Furthermore, they exhibit internal degrees of freedom in form of rotations from the orthogonal group $O(\text{dim}(\mathbf{E}_i))$ as these transform a given orthonormal basis into another valid one.

Path and Grid Graphs

The path graph P_N consists of vertices $V = [N]$ and edges $E = \{(i, i+1) \mid 1 \leq i < N\}$. The eigendecomposition of its Laplacian is e.g. given by [Spielman \(2009\)](#).

$$\lambda_k = 2 \left(1 - \cos \left(\frac{\pi k}{N} \right) \right), \quad \psi_k(i) = \pm \cos \left(\frac{k\pi}{N} \left(i + \frac{1}{2} \right) \right). \quad (2.8)$$

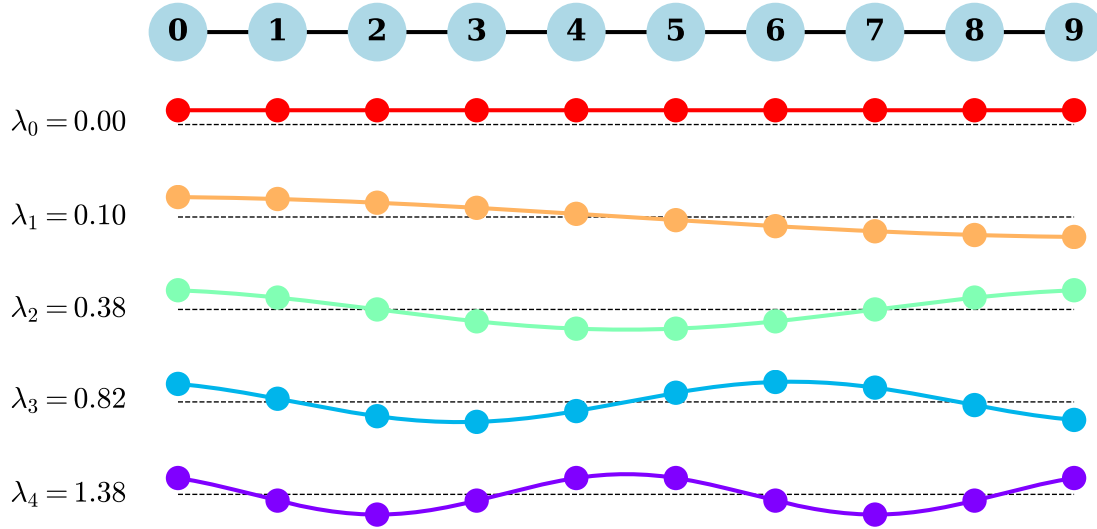


Fig. 2.3 The eigenvalues and eigenvectors of the Laplacian of the path graph P_{10} .

A grid graph $G_{H,W} = P_H \times P_W$ is the Cartesian product of two path graphs P_H and P_W . The eigendecomposition of such a product is given by the sums of the eigenvalues and tensor products of the eigenvectors of the constituent two path graphs ([Biyikoglu et al., 2007](#)). The spectrum of the grid graph is given by:

$$\lambda_{k,l} = \lambda_k + \lambda_l, \quad \psi_{k,l}(i, j) = \psi_k(i) \cdot \psi_l(j)$$

2.3.2 Laplacian Positional Encoding

Motivation

We have seen that the Laplacian eigenvectors (cf. [Figure 2.3](#)) of n -dimensional grid graphs are trigonometric functions – remarkably similar to those used in the original transformer positional encoding. As these models were used for sequences of tokens to model natural languages, we can neatly generalise this to arbitrary graphs by using

their eigendecomposition as a means of encoding positional information (Kreuzer et al., 2021).

Low-frequency eigenvectors tend to be smooth and spread across the entire graph, capturing large-scale structures, while high-frequency eigenvectors often localise on specific substructures, revealing fine-grained details based on their resonances. From this we seek to create a single vector for each node derived from this through an encoder.

Desiderata

In their *eigenfunction etiquette*, Kreuzer et al. (2021) enumerate important concerns: Eigenvectors are only unique up to their sign, or in general up to rotation in their eigenspace (if degenerate), which the encoder should ideally be invariant to. There exists a canonical ordering of the eigenspace bases following the ascending sequence of eigenvalues. However, for varying graphs, different spectral sequences will lead to the occurrence of varying frequencies at each given index of the sequence.

Definition

To mitigate this, Kreuzer et al. (2021)’s encoder treats the eigendecomposition as a multiset of pairs of eigenvector values with their corresponding eigenvalues. As this does not offer the correct invariance, they propose to apply a random sign flip augmentation to the eigenvectors during training.

$$\tilde{\psi}_l(i) = s_l \cdot \psi_l(i), \quad s \in \{\pm 1\}^k$$

The encoder takes the form:

$$\begin{aligned} \mathbf{t}_i &= f \left(\{(\lambda_l, \tilde{\psi}_l(i))\}_{l \in [k]} \right) \in \mathbb{R}^k \\ &= f_{post} \left(\sum_{l \in [k]} f_{enc} \left((\lambda_l, \tilde{\psi}_l(i)) \right) \right) \end{aligned} \quad (2.9)$$

and its output is concatenated to the projected node-features like

$$\mathbf{X}_{\text{LapPE}} = [\mathbf{X}\mathbf{W}_x, \mathbf{T}] \in \mathbb{R}^{N \times d}$$

where $\mathbf{X} \in \mathbb{R}^{N \times d_{in}}$ and $\mathbf{W}_x \in \mathbb{R}^{d_{in} \times (d-k)}$.

The simplest way to treat the set of pairs is through a DeepSet (Zaheer et al., 2018) architecture with an MLP $f_{enc} : \mathbb{R}^2 \rightarrow \mathbb{R}^k$ applied independently for each of the

k pairs. Sum-pooling across eigenvectors is then used to obtain a single vector for each node, ensuring invariance under possible reordering. Optionally, another MLP $f_{post} : \mathbb{R}^k \rightarrow \mathbb{R}^k$ can be applied after this. Alternatively, a transformer encoder could be used. Deviating from SAN, the dimensionality of the positional encoding was fixed to be equal to the number of eigenvectors k to restrict the hyperparameter space.

BasisNet

Computationally more expensive but in principle most correct is the approach taken by Lim et al. (2022). Their SignNet simply forces the encoder to be an even function of the eigenvectors to achieve invariance by summing over both possible sign flips. BasisNet goes one step further towards full $O(\dim(\mathbf{E}_i))$ invariance by wrapping a sequence model around functions of the projectors onto the eigenspaces (which are invariant under basis rotations) for each possible d_i dimensions. As these functions should also exhibit permutation equivariance, one theoretically has to use high-order tensors and IGN (Maron et al., 2019), but this quickly leads to unwieldy models which were not explored in depth in this work.

Simpler approaches were favoured as otherwise the fully correct treatment of the eigendecomposition would constitute the majority of the model’s parameters, potentially diminishing signal by the encoding modality.

2.4 Graph Machine Learning Benchmarks

Table 2.1 summarises the key characteristics of all datasets we have experimented on, organised by their origin benchmark suite.

2.4.1 Long-Range Graph Benchmark

The Long-Range Graph Benchmark (LRGB) by Dwivedi et al. (2022b) was specifically introduced to evaluate the long range interaction modelling capabilities of transformer models applied to graphs. For our experiments, we focus on two datasets from this benchmark: **Peptides-func** and **Peptides-struct**. The LRGB also contains the image-based datasets **PascalVOC-SP** and **COCO-SP** and the quantum chemistry dataset **PCQM-Contact**, which were not used in this work.

Both datasets are derived from 15,535 peptide structures, resulting in graphs with an average of 151 nodes and a large average diameter of 57.

Table 2.1 Summary of all used graph datasets.

Origin	Name	$\mathcal{Y}$	$ \mathcal{D} $	Domain	$\bar{N}$	Metric
LRGB	Peptides-func	[10]	15,535	Molecular	151	AP
	Peptides-struct	$\mathbb{R}^{11}$	15,535	Molecular	151	MAE
OGB	molpcba	[2] ¹²⁸	437,929	Molecular	26	AP
SBM	PATTERN	[2]	14,000	Synthetic	117	CE
Various	Cora	[7] ^N	1	Citation	2,708	Acc
	Citeseer	[6] ^N	1	Citation	3,327	Acc
	Pubmed	[3] ^N	1	Citation	19,717	Acc
	Cornell	[5] ^N	1	Web	183	Acc
	Texas	[5] ^N	1	Web	183	Acc
	Actor	[5] ^N	1	Wikipedia	7,600	Acc
	Squirrel	[5] ^N	1	Wikipedia	5,201	Acc
	Chameleon	[5] ^N	1	Wikipedia	2,277	Acc
RWNN	Caveman	[2]	2048	Motif	100	CE
	Cycles	[2]	2048	Motif	100	CE
Yandex	Minesweeper	[2]	1	Citation	10,000	Acc
DGK	REDDIT-MULTI-5K	[2]	5,000	Social	5,000	Acc
Custom	Coloured Diameter	$\mathbb{N}$	1,000	Synthetic	100	MAE
	CIFAR-10 Grid	[10]	60,000	Images	64	Acc

Peptides-func is a multi-label graph classification problem. Graphs are classified into 10 functional categories, evaluation uses the mean Average Precision (AP) score.

Peptides-struct is a simultaneous multiple regression task. Here, the model must predict 11 structural properties of the peptides, evaluated using Mean Absolute Error (MAE).

Criticism

Recently, criticism on the achieved performance gap by GraphGPS on LRGB has been raised by [Tönshoff et al. \(2023\)](#). This is mainly concerned with suboptimal hyperparameter tuning for baselines which GraphGPS is compared to. The performance reported by GraphGPS is still valid and this is therefore not considered problematic for our evaluation of RoPE over LapPE.

2.4.2 Open Graph Benchmark

As an additional molecular problem, we consider **ogbg-molpcba** from the Open Graph Benchmark [Hu et al. \(2020\)](#). It is a small/medium-sized 128-dimensional binary graph classification dataset which is commonly used to evaluate GNNs. Because of the importance of molecular motifs and small graph diameters, long-range interactions are not expected to be relevant for this task.

2.4.3 Stochastic Block Models

[Abbe \(2017\)](#) exposes Stochastic Block Models (SBMs) as a means to generate synthetic random graphs with community structure. The node classification dataset **PATTERN** is based on this and has been used in Benchmarking GNN ([Dwivedi et al., 2023](#)): Random pattern graphs (themselves generated by SBM) are connected to five SBM random communities. The binary label of a node then indicates membership to the pattern sub-graph. As nodes are labelled with discrete random features drawn from $\{0, 1, 2\}$, this is a purely structural task asking the model to discern fixed substructures from the background of a noisy graph.

Spectral clustering can be used to solve community detection in SBMs ([Von Luxburg, 2007](#)). The number of communities contained corresponds to the number of eigenvectors needed to resolve the problem ([Lei and Rinaldo, 2015](#)). It is therefore an interesting benchmark for the performance of transformers supplemented with spectral information.

2.4.4 Subreddits

Another purely structural task without node labels was introduced by [Yanardag and Vishwanathan \(2015\)](#) with REDDIT-MULTI-5K. The task is to classify social graphs into one of five subreddits. Graphs are given by users who are connected to each other if they have commented on the same post. The dataset is balanced and contains 5000 graphs.

2.4.5 Single-Graph Datasets

A plethora of node classification datasets derived from social networks or webpages connected with hyperlinks are used in literature as quasi-standard benchmarks to assess model performance. We experiment on the following datasets:

- the citation networks Cora, Citeseer and Pubmed ([Sen et al., 2008](#))
- the WebKB hyperlink graphs Texas, Cornell used in ([Pei et al., 2020](#))
- the Wikipedia networks Actor ([Tang et al., 2009](#)), Chameleon and Squirrel ([Rozemberczki et al., 2021](#))

all of which are helpfully bundled with `torch_geometric` ([Fey and Lenssen, 2019](#)). Node features are bag-of-words representations of the respective papers, webpages or Wikipedia pages, edges correspond to citations, hyperlinks or co-occurrences. While their small node number makes them very convenient for computationally undemanding experiments, this has also been identified as an issue as this encourages overfitting and yields high variance in performance ([Lim et al., 2021](#)).

2.4.6 Heterophilic Graphs

Homophily refers to the tendency of nodes in a graph to connect with similar ones [Hamilton \(2020\)](#). [Tang et al. \(2025\)](#) report transformer models to be advantageous in heterophilic settings.

In literature, the latter five single-graph datasets above were used to benchmark the performance of graph models on heterophilic graphs.

Recent work by [Platonov et al. \(2023\)](#) has brought up significant concerns regarding their suitability for drawing conclusions on heterophily performance. Instead, they propose (among others) a synthetic Minesweeper dataset, which requires the prediction of mines on a 100×100 grid, following the rules of the popular Minesweeper game

with mine numbers encoded as one-hot node features. This is argued to be heterophilic as mines are distributed independently at random.

2.4.7 Motif Detection

Choromanski et al. (2022) adapted the following computationally light-weight motif detection task on synthetic random graphs from Nikolentzos and Vazirgiannis (2020): Erdős-Rényi (Erdős and Rényi, 1959) graphs of size $N = 100$ are generated and connected with either a fixed motif (cf. Figure 2.4) or a random graph of same size. The task is then to detect the presence of the motif in the graph. Nodes are annotated with the top five degrees of their neighbours padded with zeroes if necessary.

A train/validation split of 75%/25% of the 2048 generated graphs is used. Reported performance is that of the best model on the validation set as there is no test set. We used the `Cycles` and `Caveman` motifs.

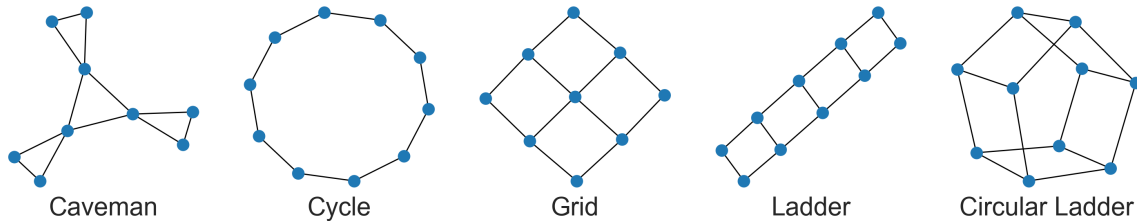


Fig. 2.4 The motifs used in the classification task. Image taken from Choromanski et al. (2022).

2.5 Benchmarking Positional Encodings

Before exploring a new positional encoding method, it is helpful to establish which known methods are advantageous in which settings. A survey of the usefulness of different positional encodings for graph neural networks was undertaken by Grötschla et al. (2024), whose main findings are summarised in the following:

The performance of several encodings (Laplacian-, random-walk-based) augmenting various model architectures (among which GraphGPS is most relevant for this work) were measured on selected tasks presented above.

Results on tasks from Benchmarking-GNN (Dwivedi et al., 2023) and LRGB (Dwivedi et al., 2022b) are heterogeneous – the effectiveness of Laplacian-based encodings is highly dependent on the specific graph dataset and task.

The aforementioned **Peptides** pair of tasks from the LRGB are an interesting test of different prediction targets derived from the same graph structures.

As expected by virtue of inclusion in the LRGB, Laplacian-based encodings are beneficial here, with strong gains achieved on the **Peptides-struct** dataset.

In contrast, on several other benchmarks, including molecular datasets from OGB, they exhibit mixed performance, sometimes pessimizing performance over models with no positional encoding.

Superpixel Graphs

For the two superpixel graph tasks contained in LRGB, there is no significant observed performance improvement from LapPE over NoPE for GraphGPS (Grötschla et al., 2024). This is in conflict with a remark made by Rampásek et al. (2022) and subsequently Tönshoff et al. (2023) who report LapPE to be rather beneficial for image superpixel graphs. Both sources obtained agreeing performance for the **PascalVOC-SP** task, so this discrepancy most likely is a question of interpretation and nuance rather than fact.

2.6 Summary

This section served to present the relevant prior works to this thesis. Two developments were covered: The synthesis of GNNs and transformers into the GraphGPS hybrid architecture; and the two orthogonal developments of RoPE and LapPE originating from the transformer positional encoding method. Having reviewed prior work and relevant datasets, we will now present the main contribution of this thesis: Applying RoPE to graph transformers and evaluating it on the benchmark tasks guided by the existing performance survey.

Chapter 3

Rotary Positional Encoding for Graphs

This chapter introduces the proposed approach of applying Rotary Positional Encoding to graphs. First, the new approach is defined, followed by a theoretical analysis of its properties and remarks on the implementation.

3.1 Definition

RoPE can be applied to general graphs by replacing the explicitly hard-coded positional features $\mathbf{T} \in \mathbb{R}^{N \times k}$ from STRING (Equation (2.5)) with the output of a LapPE encoder (Equation (2.9)). The queries and keys are rotated in the same way. This can succinctly be written in matrix notation as:

$$\{\mathbf{Q}, \mathbf{K}\}_{\text{RoPE}} = \mathcal{R}(\mathbf{T}\mathbf{\Omega}) \odot \{\mathbf{Q}, \mathbf{K}\}$$

where $\odot$ denotes per-node multiplication with the N stacked $d \times d$ rotation matrices. They are then passed to a standard self-attention mechanism:

$$\text{GraphRoPE}(\mathbf{X}, \mathbf{T}(\mathbf{A})) = \text{softmax} \left((\mathcal{R}(\mathbf{T}\mathbf{\Omega}) \odot \mathbf{Q}) ((\mathcal{R}(\mathbf{T}\mathbf{\Omega}) \odot \mathbf{K}))^\top \right) \mathbf{V} \quad (3.1)$$

This function can be substituted for the placeholder $\text{Attention}(\mathbf{X}, \mathbf{A})$ in the GPS layer (Equation (2.3)).

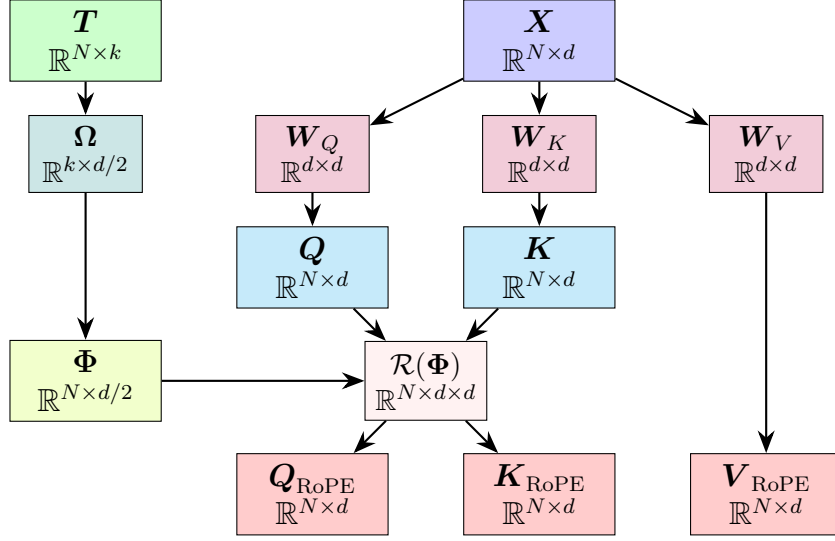


Fig. 3.1 Diagrammatic representation of RoPE/STRING for general positional features $\mathbf{T}$. Using the matrix Ω , the rotation angles Φ are derived, which modulate the key and query vectors. Values are not affected, the results are passed to an attention mechanism.

3.2 Theoretical Analysis

3.2.1 Invariance

The method directly inherits the invariance property of STRING for constant shifts in positional space applied across all nodes (Equation (2.6)). In the case of positional features derived from the eigendecomposition, the interpretation is less clear than that of shift-invariance through time or space.

Relevance of Global Shift Invariance

As the Laplacian $\mathbf{L}$ is real symmetric its eigenvectors $\{\psi_i\}_{i=0}^{N-1}$ form an orthonormal basis of $\mathbb{R}^N$. The constant vector $\mathbf{1}_N$ satisfies $\mathbf{L} \mathbf{1}_N = \mathbf{0}$ and is therefore the eigenvector associated with the smallest eigenvalue $\lambda_0 = 0$.

Orthogonality implies that for every non-trivial eigenvector ($i \geq 1$) we have $\mathbf{1}_N^\top \psi_i = 0$, i.e. all eigenvectors (except the trivial constant one) sum to zero over the node dimension.

Now suppose one were to use the raw matrix of the first k non-trivial eigenvectors used as $\mathbf{T}$ for use in RoPE. Adding a constant shift $\mathbf{c}$ to $\mathbf{t}_i$ for all i

$$\mathbf{T}' = \mathbf{T} + \mathbf{1}_N \mathbf{c}^\top, \quad \mathbf{c} \in \mathbb{R}^k \quad (3.2)$$

is therefore not a degree of freedom the mechanism needs to be invariant to.

Practically, this is not exactly applicable as the eigenvectors are not used as positional features directly but through e.g. a DeepSet encoder. There may be other, approximate ways of obtaining invariance e.g. by Taylor-expanding certain metrics based on the eigenvectors, but this has not yet been explored further.

3.2.2 Equivariance

Attention with RoPE directly inherits the desirable equivariance property as laid out by [Kreuzer et al. \(2021\)](#) for SAN as RoPE is applied node-wise using features derived from eigenvectors. The graph Laplacian and its eigenvectors are permuted in the same way as the nodes, as are the rotation angles and hence the attention output.

3.2.3 Interpretations

Rank-2 Antisymmetric Distance Matrices

Following from the above invariance property, one way of interpreting RoPE is to consider the matrix of pairwise angle differences (which modulates the actual attention kernel). This essentially establishes $d/2$ rank-2 antisymmetric distance matrices on the graph.

$$\Delta_l = \Phi_{\cdot l} \mathbf{1}_N^\top - \mathbf{1}_N (\Phi_{\cdot l})^\top = -\Delta_l^\top \in \mathbb{R}^{N \times N}$$

This is the best one can do while rotating Q and K in lockstep.

For suitable encoders, this may resemble the pairwise difference of eigenvectors used in DGN ([Beaini et al., 2021](#)) except for the fact that this difference will in general be non-zero for all pairs of nodes.

Approximating Kernels as Attention Biases

As we noted in [Vulpus and Höfs \(2025\)](#), in this way RoPE can approximate any kernel that depends only on the difference of the positional features. A similar idea had been formulated by [Liu et al. \(2023\)](#).

One can ignore the geometric notion of rotations and view them as raw intermixing of trigonometric functions applied to linear combinations of the positional features. For this, we make use of the fact that $\mathbf{W}_{Q,K,V}$ are implemented as affine transformations. This allows us to add bias terms to the attention scores independently of the node

features $\mathbf{X}$: Set $\mathbf{W}_{Q,K} = \mathbf{0}$ and keep the bias vectors $\mathbf{b}_{Q,K} \in \mathbb{R}^d$ trainable:

$$\begin{aligned}\kappa_{ij} &= (\mathcal{R}(\phi_i)(\mathbf{W}_Q \mathbf{q}_i + \mathbf{b}_Q))^\top (\mathcal{R}(\phi_j)(\mathbf{W}_K \mathbf{k}_j + \mathbf{b}_K)) \\ &= \mathbf{b}_Q^\top \mathcal{R}(\boldsymbol{\Omega}^\top (\mathbf{t}_j - \mathbf{t}_i)) \mathbf{b}_K \\ &= \sum_{l \in [d/2]} \beta_l^c \cos(\boldsymbol{\Omega}_l^\top (\mathbf{t}_j - \mathbf{t}_i)) + \beta_l^s \sin(\boldsymbol{\Omega}_l^\top (\mathbf{t}_j - \mathbf{t}_i)) \\ &\approx k_{pos}(\mathbf{t}_j - \mathbf{t}_i)\end{aligned}$$

where $\beta_l^c = (\mathbf{b}_{2l}^Q \mathbf{b}_{2l}^K + \mathbf{b}_{2l+1}^Q \mathbf{b}_{2l+1}^K)$ and $\beta_l^s = (\mathbf{b}_{2l}^Q \mathbf{b}_{2l+1}^K - \mathbf{b}_{2l+1}^Q \mathbf{b}_{2l}^K)$. This sum over sines and cosines forms a multidimensional Fourier series (Hua et al., 2024). The inherent periodicity of this series is not problematic in practice as it can be made arbitrarily large compared to the scale in positional space. With a large enough d and suitably chosen frequencies $\boldsymbol{\Omega}$, it can approximate any translation-invariant kernel. To obtain an actual even, positive-semidefinite kernel, one can apply the famous theorem by Bochner (1933) which gives a necessary and sufficient condition on the coefficients β_l^c and β_l^s when the rows of $\boldsymbol{\Omega}$ lie on the $d/2$ -dimensional lattice:

$$\beta_l^c \geq 0, \quad \beta_l^s = 0 \quad \forall l \in [d/2]$$

To not have a purely positional attention mechanism, this route may only be adopted by some attention heads or parts of the weights.

This shows that RoPE can recover the GraphiT method (Mialon et al., 2021) as their used diffusion kernel is closely related to the eigendecomposition of the Laplacian and can effectively be added to the attention scores. They explicitly mention the fact that their method does not modify the value vectors, which is also true for RoPE and leads to the following problem:

3.2.4 Graph Transformers without Semantic Features

In contrast to LapPE, which modifies the input features by concatenating/linearly mixing in the encoder output, RoPE only modifies the attention weights. For purely structural tasks where no node features are given, this limits the usefulness of RoPE for pure transformers.

Proposition 1 (Constraint for Graph Transformers with RoPE). *Applying attention modulated by RoPE to constant repeated input features gives constant repeated output node features.*

Proof. Consider $\mathbf{X} = \mathbf{1}_N \mathbf{c}^\top$ where $\mathbf{c} \in \mathbb{R}^d$ is a constant vector. The query, key, and value vectors before RoPE rotation are:

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{X} \mathbf{W}_{\mathbf{Q}, \mathbf{K}, \mathbf{V}} = \mathbf{1}_N \mathbf{c}^\top \mathbf{W}_{\mathbf{Q}, \mathbf{K}, \mathbf{V}} = \mathbf{1}_N \{\mathbf{q}, \mathbf{k}, \mathbf{v}\}^\top$$

where $\mathbf{q}, \mathbf{k}, \mathbf{v} \in \mathbb{R}^d$ are the constant repeated query, key, and value vectors. After applying RoPE, only the queries and keys become position-dependent:

$$\{\mathbf{q}, \mathbf{k}\}_i^{\text{RoPE}} = \mathcal{R}(\phi_i) \{\mathbf{q}, \mathbf{k}\} \quad \forall i \in [N]$$

but the value vectors remain unchanged: $\forall i \in [N] : \mathbf{v}_i = \mathbf{v}$.

By the definition of softmax, regardless of the modulation through RoPE, each row of the attention matrix sums to one:

$$\forall i \in [N] : \sum_{j=1}^N \mathbf{A}_{ij} = 1 \Leftrightarrow \mathbf{A} \mathbf{1}_N = \mathbf{1}_N$$

Hence we obtain

$$\mathbf{X}' = \mathbf{A} \mathbf{V} = \mathbf{A} \mathbf{1}_N \mathbf{v}^\top = \mathbf{1}_N \mathbf{v}^\top$$

□

This further shows that RoPE in general does not generalise concatenation of positional features.

3.2.5 Recovering RoPE/STRING

Path Graphs

We want to ensure that the chosen encoder architecture can give back conventional RoPE on path graphs. While exact recovery of the node index as rotary feature is impossible as we consider undirected graphs, being able to recover this up to a sign/reversal of the sequence is desirable. In our earlier report (Vulpius and Höfs, 2025), it was asserted that this is true, but some care is necessary to adapt to the actual chosen encoder architecture.

Proposition 2 (Recovering RoPE/STRING). *The DeepSet encoder Equation (2.9) with $k = 2$ and universal function approximation capabilities for f_{enc} can approximately recover the node index: $t_i = \pm i$.*

Proof. Choose a suitable f_{enc} to extract the number of nodes of the graph from the first non-zero eigenvalue. Use this to obtain the node index (up to a sign/reversal) by inverting the first eigenvector. The exact proof involving the necessary algebra is given in [Section B.1](#) in the appendix. $\square$

In practice, it is questionable whether this is necessary or achievable given the limited expressivity of the employed DeepSet encoder.

A nice property of the first non-trivial eigenvector of the path graph is that its values are bounded in $[-1, 1]$ while growing monotonically. This natural compression of the positional features is sometimes introduced artificially for RoPE to encourage length generalisation ([Chen et al., 2023](#)).

Grid Graphs

As seen in [Section 2.3.1](#) the first non-trivial eigenvectors of the path graph and thus also of the grid graph are given by:

$$\begin{aligned}\psi_{01}(x, y) &= \pm \cos\left(\frac{\pi}{H}\left(x + \frac{1}{2}\right)\right) \\ \psi_{10}(x, y) &= \pm \cos\left(\frac{\pi}{W}\left(y + \frac{1}{2}\right)\right)\end{aligned}$$

For square grid graphs with $H = W$ these belong to the degenerate eigenvalue $\lambda_{1,2} = 2 - 2\cos\frac{\pi}{H,W}$, so an arbitrarily rotated pair of these vectors may be returned by the eigensolver making it impossible to distinguish between x and y . Otherwise, an analogous argument as above shows that STRING for grid graphs can be recovered.

Training on such grid graphs while sign-flip-augmenting the eigenvectors amounts to applying the common image flipping augmentation ([Perez and Wang, 2017](#)). This can be unwanted when this symmetry is broken (e.g. in natural images).

3.2.6 Expressivity

It is widely known that GNNs are exactly as powerful as, but not more powerful than, the Weisfeiler–Lehman graph isomorphism test ([Morris et al., 2019](#)). [Kreuzer et al. \(2021\)](#) argue that even naive transformers offer universal function approximation on graphs when supplied with the adjacency matrix in some way. Making use of this they go on to show that transformers supplied with a truncated eigendecomposition of the graph Laplacian can distinguish between different graphs more easily.

A thorough analysis of the expressive power of different positional encodings is given in [Black et al. \(2024\)](#). The application of RoPE does not fit into their definition

of a concatenated absolute positional encoding (APE). In fact, it shares properties both of absolute and relative positional encodings (RPE), as the attention score is modulated based on the pair-wise differences of features derived per node.

Generally, providing the full adjacency matrix (or equivalently the full eigendecomposed Laplacian) to RoPE allows it to distinguish graphs as different adjacency matrices will lead to different rotation angles.

3.2.7 Computational Complexity

We have noted that an appealing feature of RoPE over methods requiring instantiation of the full attention score matrix is that it can be applied in time linear in the number of nodes. This property is upheld when applying RoPE to graphs, hence the time complexity stays $\Theta(Nd)$.

When applying positional features derived from the full eigendecomposition of reasonably big graphs, the number of operations to obtain this is $\Theta(N^3)$. Most often we do not care about all N eigenvectors, but only the first k eigenvectors (sorted by eigenvalue) (Kreuzer et al., 2021). A convenient method to compute the first k eigenvectors efficiently on sparse matrices is the eponymous algorithm by Lanczos (1950). This allows the computation to be performed in time $\Theta(Nk^2)$.

In combination with a linear time attention mechanism like Performer (Section 2.1.2) this would allow for entirely linear treatment of graphs using graph transformer models – enabling long range interaction modelling on very large graphs.

3.2.8 Design Choices

Encoder Architecture

The first obvious question is that of which features $\mathbf{T}$ to use to supply positional information on the graph. The options in this space have been thoroughly enumerated in the works of Kreuzer et al. (2021) and Rampášek et al. (2022).

GraphGPS offers a variety of different positional encodings, e.g. heat kernel (Andres et al., 2016) or random-walk-based ones (Dwivedi et al., 2022a).

We limited the scope of this work to LapPE as its output is most resemblant of a positional space instead of a pairwise similarity score or distance on the graph.

Hyperparameters are the number of eigenvectors k , the number of layers of the DeepSet (Zaheer et al., 2018) encoder and its MLP dimensionality. For simplicity, we choose the number of eigenvectors k to be the same as the dimension of the positional encoding and $2k$ for the MLP intermediate dimension.

The different normalisation conventions of graph Laplacians have different spectral properties; it is not clear a priori which one gives better positional signal.

Double Ω Matrices

As we are free to rotate both Q and K independently, we can consider using two matrices $\Omega_{Q,K}$ for each, which is still compatible with the linear time application. This breaks the invariance of RoPE with respect to shifts in the t -features (cf. [Section 3.2.1](#)), but allows for non-antisymmetric angular difference matrices.

Layer-wise Ω Matrices

When considering the use of RoPE in LLMs, the frequencies are typically shared across all layers or even frozen to some exponential-decay. Another alternative is to let these vary per-layer.

Ω Initialisation

When making Ω learnable, the initialisation (e.g. Xavier–Glorot ([Glorot and Bengio, 2010](#)), zeros, exponential decays ([Su et al., 2023](#)), orthogonal ([Saxe et al., 2013](#))) may influence the optimisation landscape.

3.3 Implementation

RoPE for graphs is implemented in the PyTorch Geometric library ([Fey and Lenssen, 2019](#)) on top of the infrastructure of the GraphGPS repository by [Rampášek et al. \(2022\)](#). Their framework allows for easy integration of new global attention mechanisms to be used in conjunction with local MPNNs. The implementation is publicly available¹.

As explained by [Su et al. \(2023\)](#) and summarised in [Section 2.2.2](#), the multiplication with the block-diagonal rotation matrix $\mathcal{R}(\Phi)$ is accomplished by coordinate-wise multiplication of even/odd subvectors of the key and query matrices with $\sin(\Phi)$ and $\cos(\Phi)$ of the corresponding angles. When using the existing implementation of Performer ([Choromanski et al., 2020](#)) from GraphGPS, manual masking of the value vectors (to zeros) is required to avoid the influence of dummy nodes on the attention results.

We note that the implementation has not been optimised for speed as would be advisable for a production-ready implementation. This may be achieved by providing

¹<https://github.com/cederikhoefs/Graph-RoPE>

a C++ module but was deferred to take place after the performance evaluation of the approach. For vanilla attention, a fused CUDA kernel directly computing the pairwise scalar products of $\mathbf{Q}$ and $\mathbf{K}$ modulated by $\mathcal{R}(\Phi)$ in place would be optimal. This would minimise GPU overhead by being naturally parallelisable. On reasonably large graphs, the $\Theta(N^2)$ attention mechanism would be dominating the computational effort regardless. To achieve best performance on this, FlashAttention (Dao, 2023) as supplied by PyTorch (Paszke et al., 2019) was used.

3.3.1 Spectral Cache

To allow for faster iteration and experimentation, a caching system was added to the data loading stage. The different requested eigendecompositions by possibly multiple spectral positional encodings are unified. Optionally, SciPy’s (Virtanen et al., 2020) `sparse.linalg.eigsh` implementation of the Lanczos algorithm is used on the (negated) sparse Laplacian matrix. The computed spectra are stored to disk so that repeated effort to obtain the unchanging eigendecomposition across runs can be avoided. This speeds up the startup time e.g. for many small-scale experiments where the up-front cost of computing the eigendecomposition is not negligible. Adding this as a separate, orthogonal stage of the data loading pipeline avoids the need to modify all existing datasets to include spectral precomputation.

3.3.2 Model Nomenclature

Models

With this background established, we can introduce the model nomenclature for the three main models we consider in this work:

- **GCN** A pure GCN model.
- **GT** A transformer model without local message-passing.
- **GPS** Interleaved GCN and attention.

To keep the architectures simple, a linear layer (followed by softmax for classification) was used to obtain the final prediction. Tönshoff et al. (2023) note that GCN yields significantly stronger results on **Peptides-Struct** when supplemented with deep MLP heads instead of a linear one. As they report that this leaves GraphGPS performance unaffected, the simpler linear head is used.

Positional Encodings

These architectures are supplemented with either of the following positional encodings:

- NoPE No Positional Encoding.
- LapPE Laplacian Positional Encoding. Used synonymously with concatenation of positional features.
- RoPE Rotary Positional Encoding (only possible for models containing attention).

Hyperparameters

For reference, we list common hyperparameter variable names and their meanings: The models considered have

- L layers,
- of d hidden dimensions,
- modulated by positional features treated with an encoder of L_{enc} layers,
- with dimension / derived from k eigenvectors of the graph Laplacian.

Where possible, this hyperparameter space was explored using the configurations supplied in the GraphGPS repository as starting points. The more recent OpenGT² paper Tang et al. (2025) offers additional configurations that were also used for reference.

3.4 Summary

Using RoPE on graphs gives a new modality of positional encoding that is equivariant to graph permutations and can be applied in linear time. In theory, this can recover well-known positional encodings for special graphs like path and grid graphs. We have reviewed the implementation and different models which will be experimentally evaluated in the following chapter.

²<https://github.com/eaglelab-zju/OpenGT>

Chapter 4

Experiments

4.1 Overview

This chapter contains the evaluations of the proposed approach. Unit tests were performed to ensure the implementation behaves as expected. Molecular tasks are used to compare the performance of RoPE against that of concatenated positional encodings. Purely structural tasks might profit more from the new modality and are hence used for further analysis. Finally, synthetic experiments answer basic questions about the observed results in a controlled setting.

General Remarks

Unless stated otherwise, the Adam ([Kingma and Ba, 2014](#)) optimiser was used with a learning rate of 10^{-3} and a weight decay of 10^{-5} . A cosine annealing schedule was used. A fraction of 10% of the total number of epochs was used as warmup. The number of overall epochs varied between tasks; early stopping was used when the validation loss did not improve for a number of epochs. This default setting was adopted from GraphGPS and yielded good results in the literature.

Reported performances were measured on the test set at the epoch with the best validation metric. Error bars in plots denote the standard error of the mean and not the sample standard deviation.

4.2 Unit Tests

Sanity Checks

To ensure the tested models are behaving as expected, we perform basic checks on a single-graph node-classification dataset. Our various architectures are randomly initialised and applied to the graph (and modifications thereof) without prior training. All models are equivariant and sensitive to the edge structure except vanilla transformers without positional encodings as well as with RoPE using zero frequencies.

Table 4.1 Results of unit tests performed on the **Cornell** graph. Sensitivity to the edge structure is tested by comparing with modified adjacency matrices $\mathbf{A}^{\text{empty}} = \mathbf{0}$, $\mathbf{A}_{ij}^{\text{random}} \sim \text{Bernoulli}\left(\frac{1}{2}\right)$ and $\mathbf{A}^{\text{full}} = \mathbf{1}_N \mathbf{1}_N^\top - \mathbf{I}$. All used models perform as expected, i.e. they exhibit equivariance and their output changes when modifying the adjacency matrix. The exception to the latter property are pure transformers (as well as **GT+RoPE** with disabled rotations).

Model	$f(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^\top) = \mathbf{P}f(\mathbf{X}, \mathbf{A})$	$f(\mathbf{X}, \mathbf{A}) \neq f(\mathbf{X}, \mathbf{A}')$
GPS+LapPE	✓	✓
GPS+RoPE	✓	✓
GPS+NoPE	✓	✓
GT+LapPE	✓	✓
GT+RoPE	✓	✓
GT+RoPE, $\mathbf{\Omega} = \mathbf{0}$	✓	✗
GT+NoPE	✓	✗

As RoPE compresses the magnitude of the positional features by passing them through a random $\mathbf{\Omega}$ matrix with small spectral radius, the numerical differences for an untrained model are less pronounced than for LapPE.

Low-dimensional Example

Figure 4.1 shows the output of attention with RoPE for varying positional features following the definition in **Equation (3.1)**. This also serves as visualisation of the Fourier series interpretation given in **Section 3.2.3**.

Learned $\mathbf{\Omega}$

The effective rank introduced by [Roy and Vetterli \(2007\)](#) is a continuous measure related to the rank of a matrix. This is invariant to the overall scale of $\mathbf{\Omega}$ as it is defined

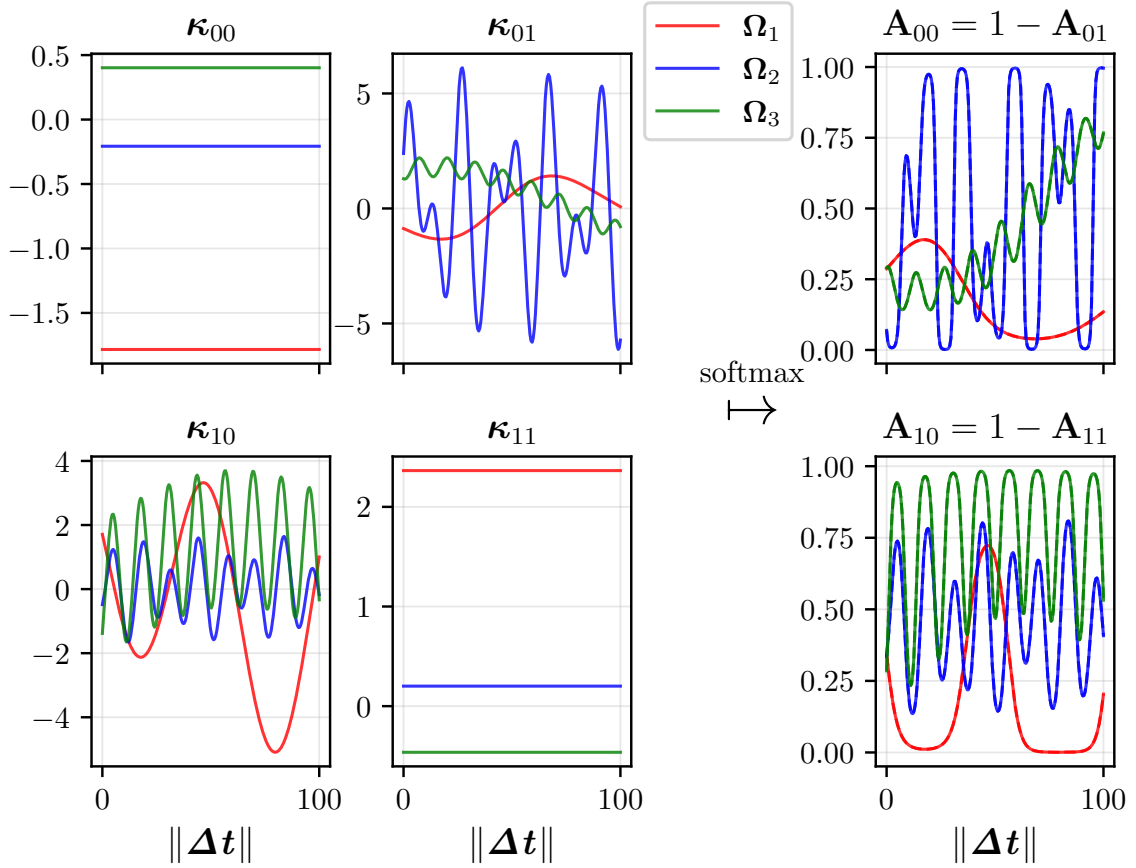


Fig. 4.1 For a RoPE attention layer of dimension $d = 4$ with $k = 4$ applied to $N = 2$ nodes, we can visualise the attention scores as a function of the positional feature difference Δt . As the rows of the three sample matrices Ω point in random directions, we expect linear combinations of sinusoids with different frequencies when Δt is scaled along a unit-norm direction. The diagonal elements of κ are of course constant as the positional feature difference between a node and itself is zero. After the application of softmax, non-harmonic (but still periodic) functions are seen as the resulting attention weights.

as the exponentiated entropy of the distribution of normalised singular values. The scale of the features $\mathbf{T}$ is not fixed as they are the output of the nonlinear Laplacian positional encoder.

By plotting this metric over the training epochs, one can qualitatively compare initialisation schemes for $\mathbf{\Omega}$. In this example case the maximum rank is 4 as $\mathbf{\Omega} \in \mathbb{R}^{4 \times 16}$.

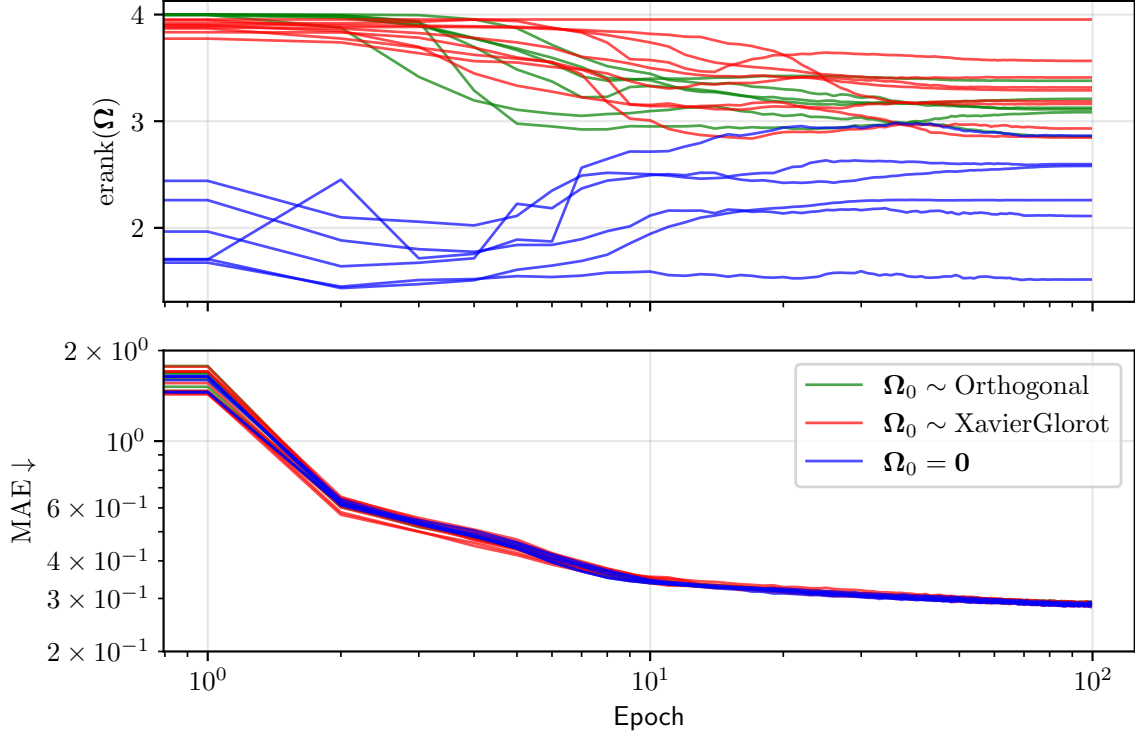


Fig. 4.2 Evolution of $\text{erank}(\mathbf{\Omega})$ over the training epochs for different initialisation schemes on the **Peptides-struct** task. The used model is one layer of GraphGPS with RoPE on LapPE with $k = 4$ and $d = 32$. Orthogonal initialisation starts exactly at rank 4. Random initialisation yields matrices with effective rank close to that while zero initialisation is close to rank 0. All the effective ranks seem to converge as the optimisation progresses. Notably the effective rank of the latter stays lower than that of the former across multiple runs. From the definition of the effective rank it is clear that it is bounded below by 1 even for the zero matrix.

Interestingly, the regression loss for all schemes behaves near-identically despite the difference in initialisation.

Evidently the initialisation has lasting influence on $\mathbf{\Omega}$, but the final performance as measured by the test loss is not affected. This leaves the question of the optimal dimensionality of the positional features unanswered.

Inspecting the norm of $\mathbf{\Omega}$ after training shows that it is non-zero – RoPE is hence used to modify the attention mechanism.

4.3 Molecular Tasks (Peptides & molpcba)

Rationale

Experiments on LRGB’s **Peptides** datasets constituted the starting-off point of this work as some evaluation on **Peptides-func** had already been done in [Vulpus and Höfs \(2025\)](#). To obtain an initial understanding of the performance of RoPE, parameters were chosen based on the GraphGPS configurations, only swapping concatenation for RoPE as a mode of positional encoding.

Results

Table 4.2 Summary of the performance of GPS on the **Peptides-func** dataset. A $d = 96$ -dimensional model with $L = 4$ layers and 16-dimensional positional encoding with $k = 10$ as reported by [Rampásek et al. \(2022\)](#) is used. In this case only, to keep comparability with their reported results, **GatedGCN** ([Bresson and Laurent, 2017](#)) is used as the local model. RoPE adds few parameters and improves the performance over NoPE, albeit not as much as LapPE.

Encoding	AP	#Parameters
NoPE	62.5% $\pm$ 0.6%	506602
LapPE	64.9% $\pm$ 0.2%	504442
RoPE	64.1% $\pm$ 0.2%	507994

Enabling LapPE nominally reduces the parameter count as the fixed number of dimensions is shared between it and the atom encoder which is more parameter-intensive. With $L_{enc} = 2$, the encoder contains $2 \times (2k) + (2k)$ parameters, while the atom encoder dictionary requires substantially more parameters to treat each atom type. Most of these parameters are unused as the considered molecules only contain a small subset of all 119 possible chemical elements.

[Table 4.2](#) gave the initial confirmation that RoPE works as a way of using Laplacian positional encodings, though not as efficiently as concatenating them.

Spectral Dimension

GraphGPS ([Rampásek et al., 2022](#)) performed extensive hyperparameter tuning.

A natural next question to consider is hence what influence the number of encoded eigenvectors k (also corresponding to the dimension of the positional features) has

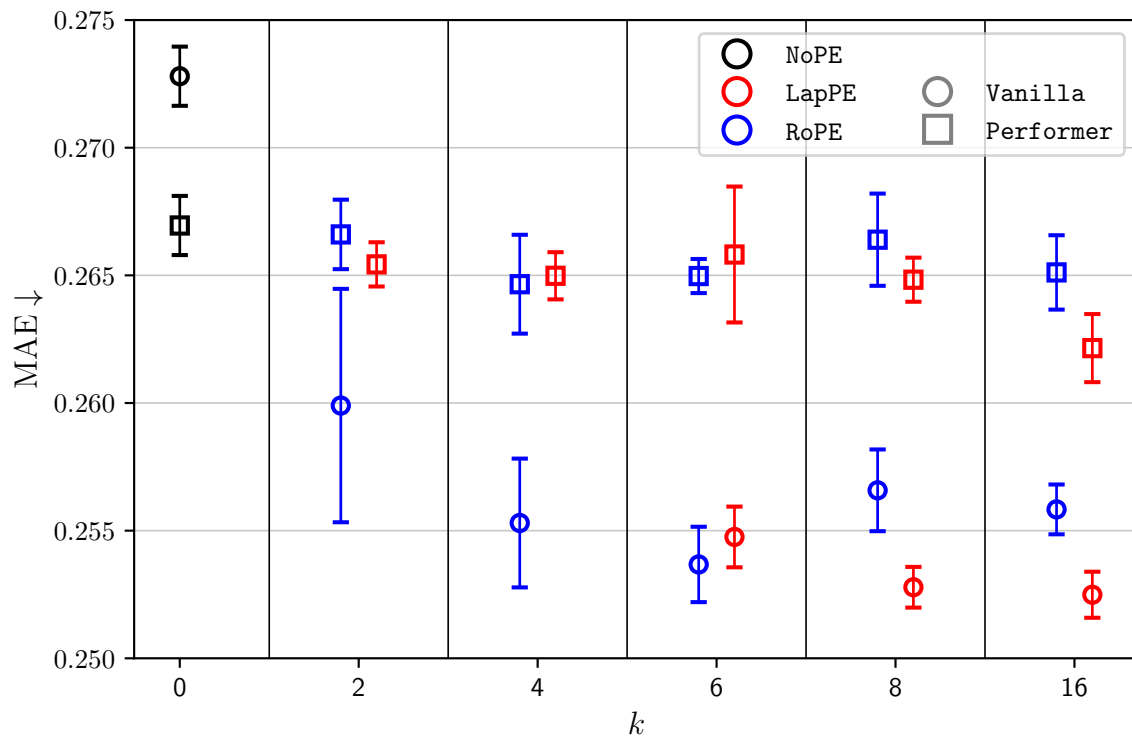


Fig. 4.3 Performance on the **Peptides-struct** dataset. **LapPE** with small k faces convergence issues and falls outside the plotting area with high variance. **RoPE** exhibits reduced variance and better performance for higher k , as does **LapPE**. Using **Performer** instead of full attention leads to a performance drop. For this linear attention mechanism, both positional encodings exhibit very similar behaviour with varying k . At $k = 16$, **LapPE** achieves the best performance overall.

on performance. [Figure 4.3](#) shows the performance of the various models on the `Peptides-struct` regression dataset.

Exchanging vanilla attention for **Performer** leads to an expected performance drop, with very similar performance of **LapPE** and **RoPE**.

Discussion

Overall this means that **RoPE** is a viable method of using spectral positional information in a graph transformer, across both regression and classification tasks on the same molecules. Allowing for higher-dimensional positional features makes the model more expressive and indeed performance improvements are observed.

4.3.1 Non-Long-Range Tasks

Rationale

OGB’s `molpcba` is a medium-scale molecular multi-label classification task ([Hu et al., 2020](#)). Numerous classes are present, forcing the model to learn representations that generalise meaningfully, e.g. of chemical motifs. The task primarily stresses local chemical interactions rather than long-range dependencies. This makes it suitable to document that **RoPE** does not pessimise performance of **GraphGPS** for such tasks. While **GraphGPS** opted for random-walk-based encodings for this task over **LapPE**, [Grötschla et al. \(2024\)](#) found them to perform better.

Results

RoPE’s performance on these tasks is again not superior to that of **LapPE** (cf. [Figure 4.4](#)). On the other hand, the method shows a consistent trend in performance versus model capacity.

Discussion

A performance gap in comparison to [Grötschla et al. \(2024\)](#) was expected as they benchmarked considerably larger models.

The results on **LapPE** underline the need for hyperparameter tuning; concatenated encodings perform better for small k which cannot be derived from first principles. The monotonicity in improvements when supplying growing amounts of spectral information is encouraging.

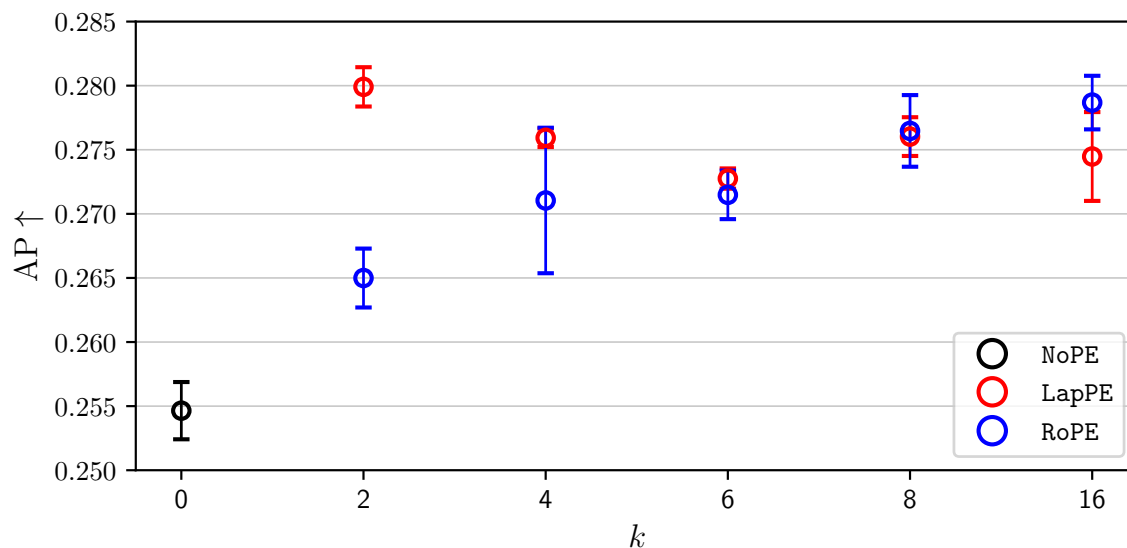


Fig. 4.4 Performance comparison of GPS models on the molpcba dataset. LapPE surprisingly performs best for small k and then drops off. RoPE exhibits monotonic improvement with growing k , but does not surpass concatenation.

4.3.2 Conclusion

The molecular tasks considered so far required the treatment of both semantic and structural information at once. In the following sections, a more thorough investigation of RoPE on clear-cut realisations of each task type is performed. This helps in discerning the usefulness of RoPE for these two special cases in isolation.

4.4 Structural Tasks (PATTERN & REDDIT-MULTI-5K)

Rationale

PATTERN is a purely structural task where spectral information plays an important role. A clear trend of improving performance with more eigenvectors is expected from Abbe (2017)’s theoretical exposition. As five communities are used, we expect an increasing trend in performance up to that number of eigenvectors. After this no further improvement is expected.

The task’s non-identical random features demonstrate the expressivity of pure GT models without semantic information. The node features are not strictly identical, and hence the problem described in Section 3.2.4 does not apply.

In [Section 3.2.8](#) several variations of the RoPE design were considered. These will also be evaluated on the PATTERN dataset in the following sections.

Results

[Figure 4.5](#) exactly matches the expected behaviour, with a small penalty for RoPE over concatenation and diminishing returns for growing k .

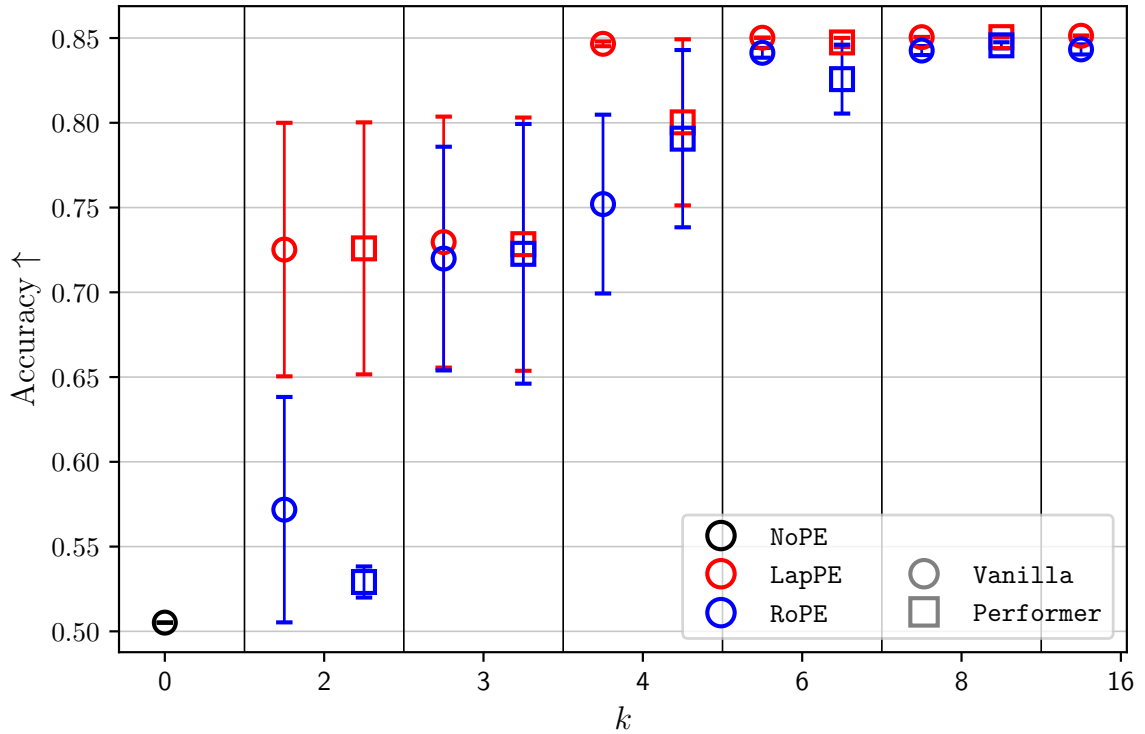


Fig. 4.5 Accuracy of GT on the PATTERN dataset. Both LapPE and RoPE exhibit monotonic improvement in mean accuracy and reduced variance until $k = 5$. Concatenation performs consistently better, but the gap shrinks with growing k . There is no performance penalty for using linear attention for this task. A graph transformer without positional encodings is not able to learn the task.

Discussion

The achieved performance of both encodings for $k > 5$ is in agreement with data by [Dwivedi and Bresson \(2020\)](#). GraphGPS reported only marginally higher performance with message passing enabled ([Rampásek et al., 2022](#)).

This is due to the fact that this task can be solved from spectral information only without exact local information transfer along edges. The lack of a performance drop

when using linear attention is also explained by the fact that there is no semantic information to be refined using full softmax attention.

4.4.1 Encoder Design

For this dataset, the symmetrically normalised Laplacian gives performance gains over the unnormalised Laplacian. This is in line with theoretical and empirical results in literature noting benefits from normalisation in analysing SBMs (Deng et al., 2021). In other examined datasets, no clear trend for the performance of the unnormalised or symmetrically normalised Laplacian is evident.

As there is no benefit from deeper DeepSet encoders visible in Figure 4.6, it is evident that the eigenvectors are used by the model without need for sophisticated encoding in this task. This is again sensible considering spectral clustering of SBMs does not require expressive functions applied to the eigenvectors (Abbe, 2017) either.

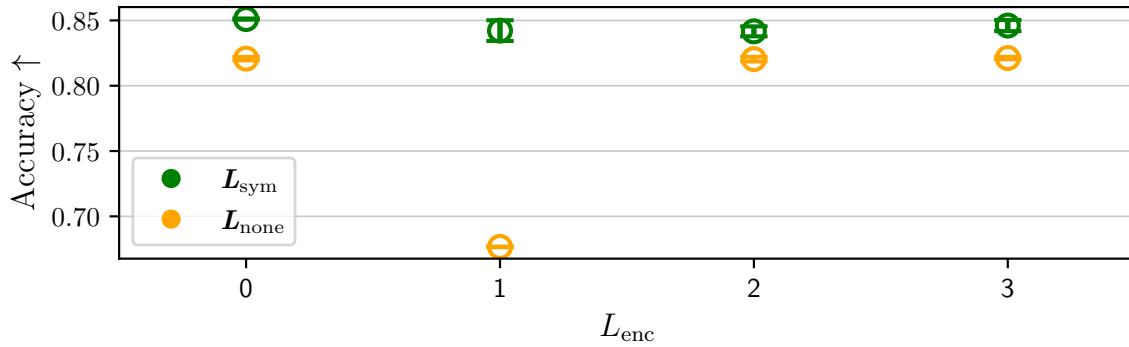


Fig. 4.6 Effect of the encoder depth L_{enc} and Laplacian normalisation for GT+RoPE performance on PATTERN (5 seeds). Normalising the Laplacian yields significantly better performance. A deeper DeepSet encoder does not improve the accuracy for this task.

4.4.2 Shared Omega Matrices

One may use different Ω matrices for the query and key matrices or across layers. For GT models, neither variation yields any performance gains (cf. Figure 4.7). An increase in variance is observed for the model with the most parameters, two separate per-layer Ω matrices.

This result suggests that additional capacity in deriving the angles from positional features is not needed. Therefore, Ω matrices are shared across layers and query/key vectors in all following experiments.

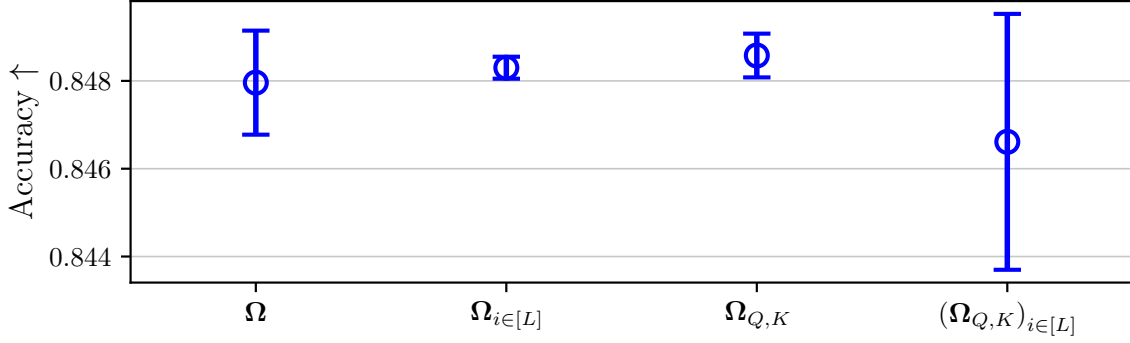


Fig. 4.7 Influence of the Ω matrix on GPS+RoPE performance on the PATTERN dataset (5 seeds). No significant differences appear between shared and non-shared Ω for either layers or query/key vectors.

4.4.3 Expressivity Breakdown for GT+RoPE

Rationale

The REDDIT-MULTI-5K classification task does not contain any node labels, not even randomly drawn ones. Instead, the nodes are annotated identically with a constant which leads to the realisation of the problem discussed in Section 3.2.4. We expect the prediction of a pure graph transformer with RoPE to be random among the five classes, corresponding to an accuracy of $\frac{1}{5}$.

Results

GT+LapPE achieves an accuracy of $51.4 \pm 1.0\%$, while GT+RoPE achieves $20.2 \pm 0.0\%$, failing as expected. This experiment solely serves to showcase this failure case. The lacking performance of the graph transformer with LapPE compared to reported results for GCN (Job et al., 2024) was not further investigated.

4.5 Single-Graph Datasets

Rationale

To obtain data on the usefulness of RoPE on the various single-graph classification datasets from Section 2.4.5 commonly used in the literature, matching configurations by Tang et al. (2025) were trained for all architectures and encodings.

As stated by [Lim et al. \(2021\)](#), the small number of nodes in these datasets leads to high variance in performance, mostly explained by the misclassification of single nodes. A large number of reruns was thus performed to obtain [Table 4.3](#).

Results

Table 4.3 Node classification accuracies (%) on various single-graph datasets (20 seeds). Many performance differences from introducing LapPE or RoPE were not statistically significant, despite the large number of runs.

Dataset	GT			GPS		
	NoPE	LapPE	RoPE	NoPE	LapPE	RoPE
Actor	37.4 ± 0.2	37.2 ± 0.1	37.5 ± 0.1	36.8 ± 0.2	36.6 ± 0.1	36.7 ± 0.2
Chameleon	48.6 ± 0.2	48.4 ± 0.3	48.7 ± 0.2	47.8 ± 0.3	49.2 ± 0.3	47.8 ± 0.2
Citeseer	50.8 ± 0.3	50.9 ± 0.2	50.9 ± 0.2	61.1 ± 0.3	62.6 ± 0.4	61.1 ± 0.5
Cora	52.1 ± 0.4	52.3 ± 0.3	52.0 ± 0.3	70.7 ± 0.3	70.6 ± 0.4	69.9 ± 0.4
Cornell	68.1 ± 0.7	70.3 ± 0.7	70.1 ± 0.9	70.1 ± 0.6	69.7 ± 0.8	69.5 ± 0.8
Pubmed	69.5 ± 0.3	69.5 ± 0.2	69.6 ± 0.2	74.5 ± 0.1	74.5 ± 0.1	74.2 ± 0.2
Squirrel	35.8 ± 0.1	35.5 ± 0.1	35.4 ± 0.2	35.2 ± 0.2	34.9 ± 0.2	35.0 ± 0.1
Texas	77.7 ± 0.3	76.2 ± 0.5	76.8 ± 0.4	75.9 ± 0.5	75.5 ± 0.4	76.2 ± 0.5

The achieved performance of GPS+LapPE on Cora and Citeseer agrees with that reported by [Lim \(2024\)](#). Relative improvements of GPS+LapPE over GPS+NoPE are in agreement with reports by [Tang et al. \(2025\)](#).

These data are used in a joint analysis in the following section.

4.5.1 Correlation of Effects of LapPE and RoPE

Rationale

RoPE and LapPE share the same Laplacian positional encoding as input. A correlation between the performance gains over models without positional encodings across datasets is hence expected. To confirm this, one can measure the correlation between the accuracy gains of RoPE and LapPE over NoPE. This is methodologically not ideal as the accuracies need not be comparable, but defensible as all of these pose similar node-classification tasks.

Results

There exists a strong correlation between the performance gains of RoPE and LapPE (cf. Figure 4.8) for GT without message passing. The same holds when considering relative performance increases normalised by the performance of NoPE.

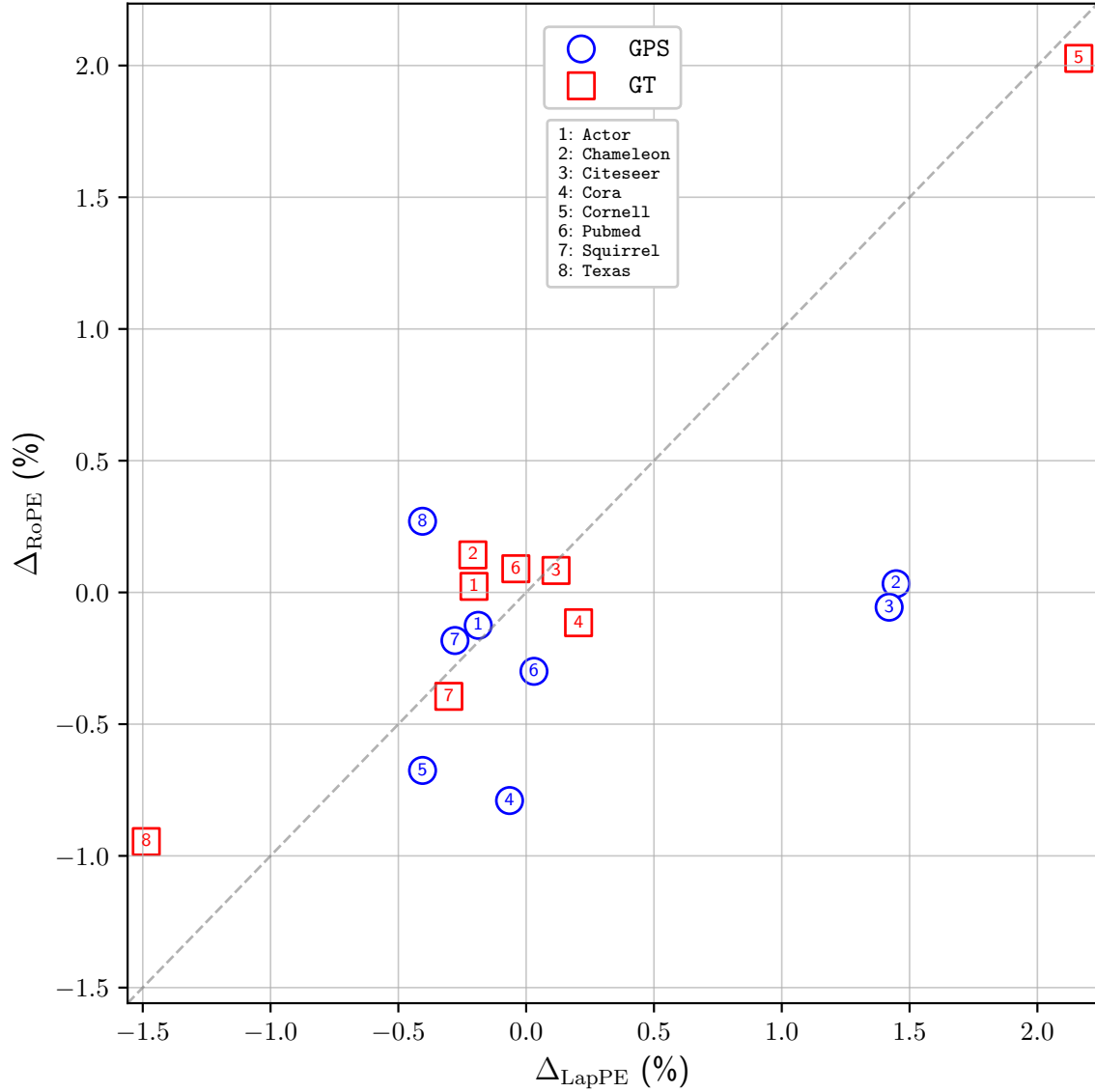


Fig. 4.8 Correlation between gains through LapPE and RoPE for eight single-graph datasets. For GPS models, there is no significant correlation between the performance gains. In graph transformers, a significant, strong correlation of $\rho = 0.967$ is observed.

Discussion

This makes sense as a pure transformer without positional encodings would be oblivious to the graph structure – adding positional information through any means is hence expected to improve performance. GPS models on the other hand already contain message passing along the graph edges, so RoPE did not give any performance gains for these.

Still, the validity of these findings is questionable as test set accuracies become discretised due to the small number of nodes. The resulting variance therefore makes many individual improvements statistically insignificant. Despite these shortcomings, the difference in behaviour between GT and GPS is still notable.

4.5.2 Hyperparameter Tuning

Cornell was chosen as a cheap example task to optimise various hyperparameters. Both the attention and MLP dropout rate of 0.5 were found to be optimal out of $\{0.1, 0.25, 0.5\}$. Only later the problem of small node counts was noted which questions the validity of the results.

4.6 Heterophilic Graphs (Minesweeper)

Rationale

As an alternative to the structural tasks, Minesweeper was chosen to test the scaling behaviour of RoPE for larger graphs. This showcases the potential end-to-end treatment of big graphs in $\Theta(N)$ time using GraphGPS and spectral encodings. The large 100×100 grid graph alleviates the problem of too few nodes which made the results on the above datasets noisy and inconclusive. In terms of spectral coordinates, it should be helpful that the underlying grid graph is unchanged for all instances.

As Tang et al. (2025) noted that Transformer models are well-suited for treating heterophilic graphs, this further serves as an investigation into performance differences between rotary and concatenated encodings in such settings.

Results

No differences are observed between the various encodings (cf. Figure 4.9). GCN and GPS models show a small but significant difference.

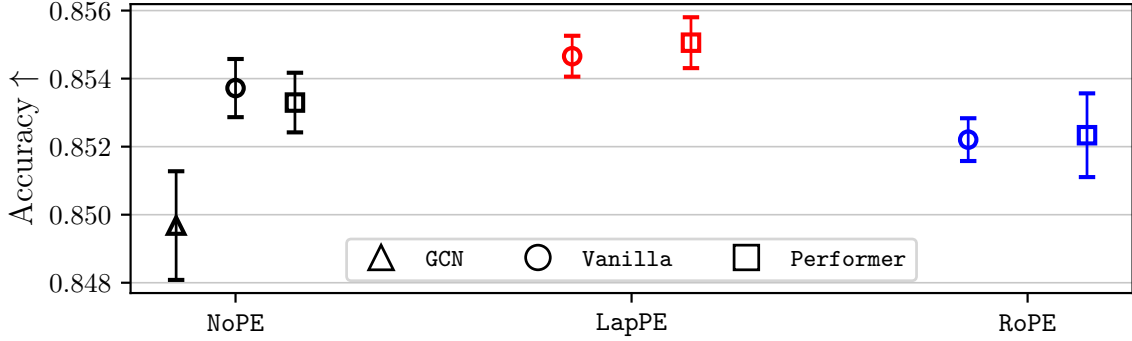


Fig. 4.9 Performance of $d = 64$ -dimensional GPS models on the **Minesweeper** task with $k = 16$ -dimensional positional features (25 seeds). Negligible gains are observed for **LapPE** over **NoPE** for this task. Applying **RoPE** leads to a small performance drop. Despite the large number of runs, statistical significance is not achieved. There is no performance difference between **Performer** and full attention. Adding attention improves marginally upon **GCN**.

Discussion

The obtained identical results make sense when considering that the **Minesweeper** task is a very local problem. Clearly, the top few eigenvectors of a grid graph of this size vary too slowly to resolve immediate neighbourhoods which makes them irrelevant for this task. Minimal gains when including attention may be attributed to increased model capacity. Consequently, despite the heterophilic nature of the graph labels, **RoPE** and **LapPE** are not advantageous. This is expected for this task as the graph-structure is not informative in this task, so encoding it is not necessary.

A direct comparison with the results e.g. for **GCN** reported in the original paper is not meaningful. They use substantially higher hidden dimensions ($d = 512$) and perform grid search across depths up to $L = 5$ layers and thus achieve higher performance (Platonov et al., 2023).

4.6.1 Lanczos' Algorithm

Rationale

Computing the full eigendecomposition for the above large grid graph ($N = 10^4$) has cost $\Theta(N^3)$ and is therefore infeasible. Instead, we would like to use Lanczos' algorithm to compute the top k eigenvectors in linear time. To ensure using this numerical method does not introduce material errors, the quality of its results was measured as follows: Figure 4.10 shows the difference of the approximate eigenvectors and their

projection onto the eigenspace spanned by the analytical eigenvectors. These are known for the grid graph (cf. [Section 2.3.1](#)).

Results

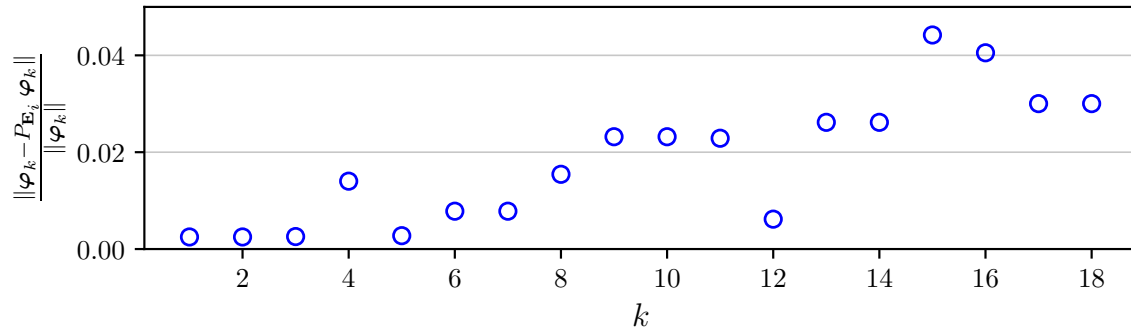


Fig. 4.10 Relative norm of the residual of the eigenvectors over the analytical eigenvectors on the **Minesweeper** task.

The results are numerically nearly identical with slight differences for higher eigenvalues while reducing computational cost by orders of magnitude.

In hindsight, as inclusion of spectral information did not make a difference in performance for **Minesweeper**, this was not the best showcase for Lanczos’ algorithm. It was used on most other datasets without any noticeable degradation of performance.

4.7 Motif Detection (Cycles & Caveman)

To explore the interplay of the hidden dimension d and spectral dimension k as well as other hyperparameter choices, we performed an extensive grid search. The detection of **Cycles** and **Caveman** (cf. [Section 2.4.7](#)) are computationally cheap tasks which therefore were used for this purpose.

4.7.1 Hyperparameter Sweeps

[Figure 4.11](#) shows the performance of differently-sized models on **Caveman** with varying amounts of spectral information. As expected, bigger models have more capacity and were able to achieve higher validation performance. The training of the largest models on the other hand failed to converge, achieving poor mean performance with high variance.

In other experiments, the learning rate was varied in the range of $\eta \in \{10^{-3}, 10^{-4}, 10^{-5}\}$. The best results were obtained with our default of $\eta = 10^{-3}$.

A similar analysis has been carried out for **Cycles** with the same conclusions which are not shown here.

4.8 Coloured Subcomponent Diameter

Rationale

RoPE works well when presented with explicit, repeatable coordinate spaces on large-scale datasets, e.g. grids of patches of an image. Real-world graphs exhibit much more complex structures which may lead to suboptimal performance.

In order to test the stability of RoPE and LapPE under less straightforward spectral coordinates, the following synthetic task was devised: A grid graph of size $n \times n$ is generated. Nodes have binary features which are independent Bernoulli random variables with parameter p . The task is to predict the diameter of the largest connected component of coloured nodes (i.e. nodes with feature $X_i = 1$) in the graph. A train/validation/test split of 80%/10%/10% of the 1000 generated graphs is used. The L1 regression loss/MAE is used for training and evaluation of the models.

4.8.1 Rewiring Robustness

Concerningly, **GCN** outperforms the strictly more general **GPS** model on this task. Even enabling the attention mechanism of the **GPS** model with zero-initialised parameters leads to a significant performance drop. This may be attributable to modified learning dynamics leading towards suboptimal local minima.

A strong, uniform performance drop for all models is observed when any rewiring of the graph is performed, regardless of the non-zero rewiring probability. This prompted the development of the image classification task in the next section.

4.8.2 Sign Flip Invariance

We have alluded to the fact that our models should overall be invariant to the sign of the eigenvectors. Random sign flips are introduced as augmentation during training so that models are incentivised to learn this invariance approximately.

As this synthetic dataset is a regression task, we can test this by performing inference on trained models for all possible sign flips of the eigenvectors, measuring the

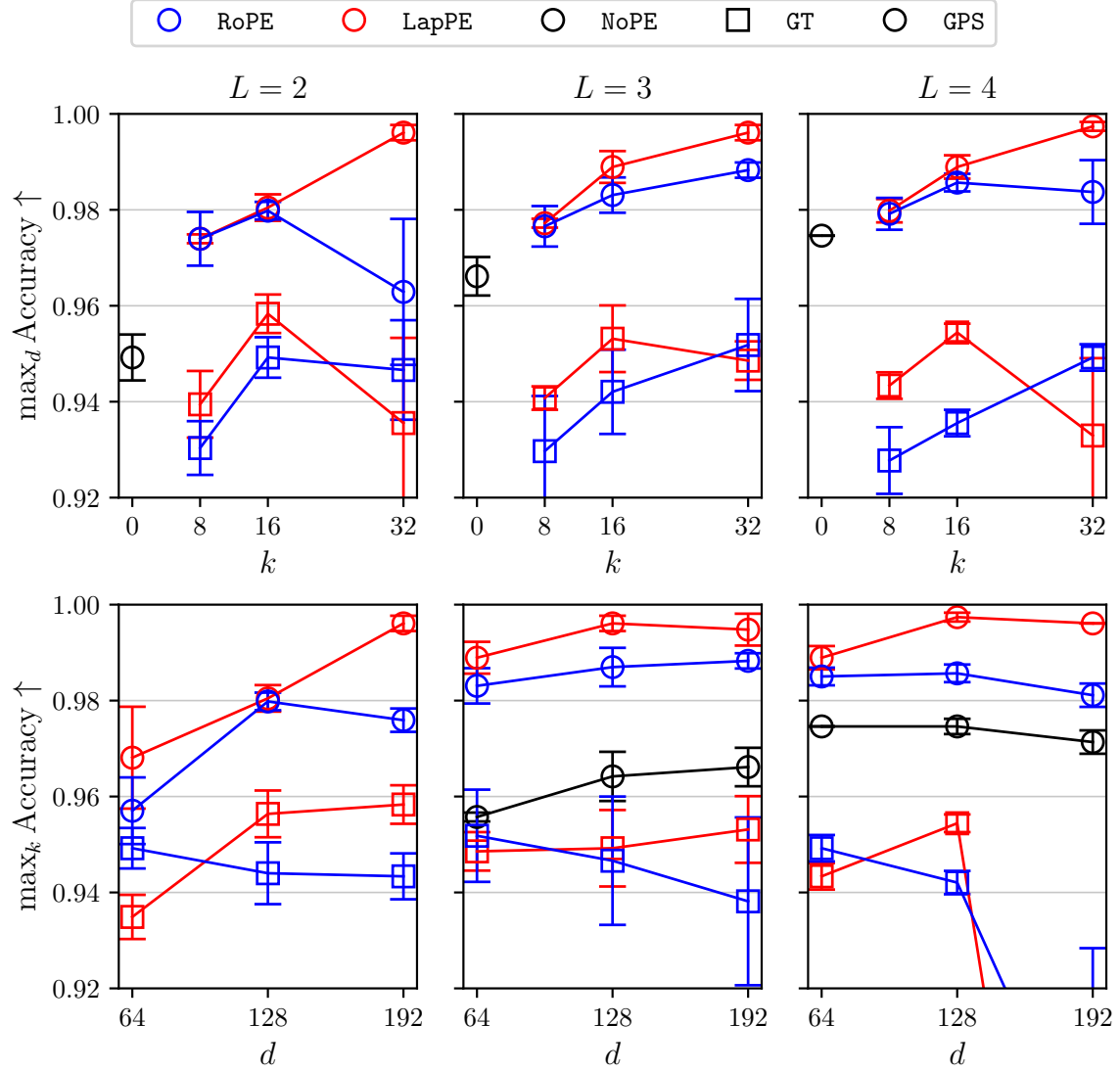


Fig. 4.11 A comprehensive grid search was performed on **Caveman** to understand the effects of varying L , d and k . For these line plots, the best performing model along the other respective hyperparameter is used. There is a consistent performance gap between **LapPE** and **RoPE** models who both outperform **NoPE**. **GPS** performs better than graph transformers. Both increases in d and k lead to monotonic improvements in performance for both encodings. Especially for deeper models ($L = 4$), the largest models suffered from training instability.

absolute error of each prediction. The predictions of GraphGPS using both RoPE and LapPE are invariant to sign flips of the eigenvectors, little variance is observed when enumerating all possible sign flips.

4.9 Grid Graphs (CIFAR-10)

To investigate the performance drop of our models under rewiring of grid graphs, we consider the CIFAR-10 dataset. The image data supply some semantic information to be mixed with positional information through RoPE.

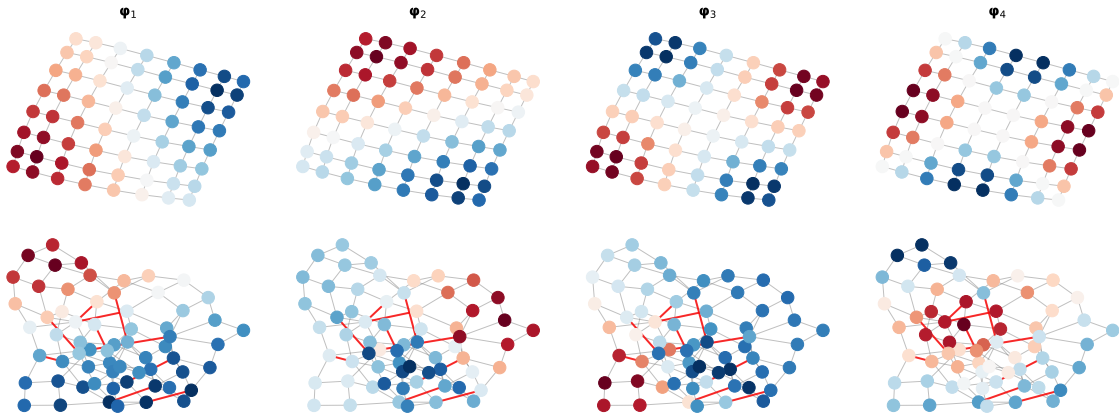


Fig. 4.12 Spring-force layouts of a grid graph with and without 10 random added edges (in red). The first four non-trivial eigenvectors of the graph Laplacian are shown as a heatmap.

4.9.1 Recovering ViT

Rationale

Following the analysis in [Section 3.2.5](#), Vision Transformers (ViT) can be seen as special cases of graph transformers applied to grid graphs. To test this, CIFAR-10 images are split into patches of size 4×4 with the superimposed 8×8 grid graph structure connecting adjacent patches. This is notably different to the common superpixel clustering approach to formulate CIFAR-10 classification as a graph task (e.g. by [Dwivedi et al. \(2023\)](#)).

For square grids, the second and third eigenvectors are degenerate, giving an internal degree of freedom with a rotation matrix in the subspace spanned by ψ_1 and ψ_2 (cf. [Figure 4.12](#)).

Directly supplying x and y as rotary or concatenated coordinates should give the same performance as a plain ViT (with linear readout instead of an additional classification token). Inputting the first three eigenvectors allows determining whether the learned DeepSet encoders are able to recover equivalent performance to this from spectral features.

Finally, to test the robustness of RoPE against non-straightforward eigenvectors supplied as positional coordinates, a rewiring of the graph is performed. Here we add a configurable number of random edges to the base grid graph.

Results

The results are shown in [Figure 4.13](#). From this experiment several conclusions can be drawn: Distorting the grid structure incurs a uniform performance drop across all methods in this task as well; this is evidence against excess instability of RoPE under slight spectral coordinate changes. For GT models, combining concatenation and rotation surpasses LapPE.

Discussion

The gap between explicitly provided x and y versus learned spectral features is more pronounced for RoPE – this may be an explanation for its worse performance in other graph tasks. Learning of robust positional features of graphs through the RoPE mechanism may be impeded.

4.10 Summary

This chapter presented the empirical evaluation of the proposed approach. Unit tests confirm the correct implementation of RoPE for graphs. Diverse experiments show that RoPE consistently improves performance over models without positional encodings. Linear attention can be employed, as can linear-time spectral decomposition algorithms. The scaling behaviour regarding positional encoding and model dimension has been investigated. The regularity of the underlying graph structure has been considered as a possible factor in the performance of RoPE.

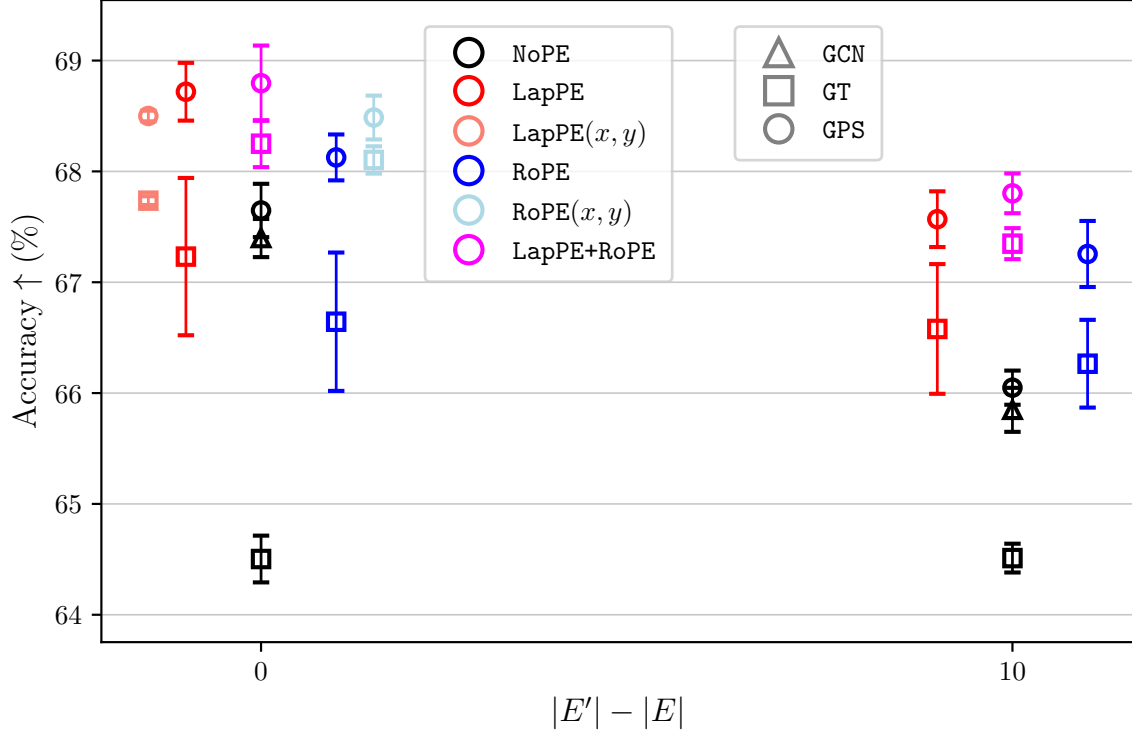


Fig. 4.13 Mean test accuracies with standard errors of the mean for different models on CIFAR-10 grid graphs. $k = 4$ eigenvectors are used with a DeepSet encoder for the positional features.

GT+NoPE constitutes the baseline as it is a pure function of the multiset of patches and hence unaffected by the added edges.

Across both architectures, substituting RoPE for LapPE leads to a performance drop, but adding it on top increases performance (especially noticeable for GT).

RoPE applied to explicit x and y coordinates achieves significantly stronger performance even for GT. Concatenating x and y (dubbed LapPE($\mathbf{x}, \mathbf{y}$) for consistency) also provides a performance gain (only for) GT, but a smaller gain.

Adding 10 random edges leads to a uniform performance degradation across all methods. The only change in order is that GPS+NoPE and GCN underperform GT+LapPE and GT+RoPE.

Chapter 5

Conclusion and Future Directions

After presenting the main results of this thesis, the following section comments on its broader relation to the field of graph machine learning. Key strengths and limitations are mapped for reference. The chapter closes the thesis by discussing open questions to be treated in future work and ways forward.

5.1 Impact

This thesis investigated a novel method for encoding positional information within graph transformers. The approach was inspired by the success of Rotary Positional Encodings in Large Language Models. While its applicability to graph-structured data is not immediately obvious, experiments demonstrate its potential as a useful component for graph transformers in certain settings.

Considering all experiments, the results are mixed and not necessarily conclusive. The lack of performance when compared to concatenated positional encodings remains to be investigated for better understanding.

Looking forward, the machine learning field is increasingly moving towards large-scale, pre-trained models that leverage substantial computational resources. Adapting these powerful transformer architectures to the graph domain is therefore a crucial research avenue. This work contributes to this effort by exploring a new modality to provide a soft inductive bias for graph structures within the transformer framework. However, a significant challenge for graph pre-training is the absence of massive, standardised datasets comparable to those in language modelling.

5.2 Strengths and Limitations

Strengths

Theoretical motivation for applying RoPE to graphs has been provided. Some proven properties have now explicitly been documented.

This work yielded several datapoints on the performance of RoPE on various graph datasets. A reusable implementation on top of the popular GraphGPS framework is provided. This allows for future work to build upon what has been implemented in the course of this project.

Some understanding regarding the failure mode of RoPE on graphs has been gained by formulating custom synthetic tasks.

Limitations

The experiments have been focused on small to medium-scale datasets. There is no clearly understood failure mode yet: Are the eigenvector-derived position spaces too complex to be used as positional features for RoPE?

For path graphs, passing eigenvectors as input to RoPE amounts to feeding oscillatory, higher-order sinusoids through rotation matrices. This modality may simply not be appropriate.

Getting equivalent performance to that of RoPE in LLMs might also be a question of scale (both of models and data). This is a clear limitation of this work accepted to allow for fast iteration and broader scope without waste of computational resources.

Empirically, faster/more stable training is reported for RoPE in LLMs. This also appears to be the case on graphs and may well be motivated by the numerical conditioning through RoPE, but this was not quantified due to the lacking overall performance of the method.

Deciding early on to fix the dimensionality of the positional encoding to the number of eigenvectors might have been too restrictive. This limited the expressivity of the DeepSet encoder prematurely, as it might be necessary to scale this while keeping the number of treated eigenvectors low.

5.3 Open Questions and Future Work

The question of how to best supply positional information to graph transformers is still open and research is ongoing. It has not definitively been settled how spectral

information should be encoded (Bo et al., 2023) or whether principally spectrally based architectures might be more appropriate (Garg, 2024).

Furthermore, treating the eigendecomposition as a bag of pairs ignores the natural ordering introduced by the eigenvalue magnitude. Adhering to this ordering without fully adopting BasisNet (Lim et al., 2022) might be beneficial for better model performance.

Future work on RoPE on graphs might focus on the following:

Size Generalisation

Do size generalisation benefits reported for RoPE (Su et al., 2023) carry over to graphs when training on a different distribution of graph diameters than the test set? Eigenvectors are inherently normalised across arbitrarily large graphs – this may bring benefits or be a reason for diminished performance.

Linear-Time Treatment

Does the feasible $\Theta(N)$ treatment bring benefits to machine learning on very large graphs? The interaction with graph-adapted linear attention mechanisms like Exphormer (Shirzad et al., 2023), Graph ViT/MLP-Mixer (He et al., 2023) or even simpler linear attention kernels in the GraphGPS framework (Katharopoulos et al., 2020) should be investigated.

Different Positional Features

The focus on Laplacian-based positional features might have been too narrow. Due to its modularity, LapPE can be replaced with any other positional encoding offered by GraphGPS.

Vellingker et al. (2022) introduce resistive embeddings whose pairwise distances reproduce commuting times in the norm of pairwise differences. While these are very high-dimensional, dimensionality reduction techniques may make these feasible for use with RoPE. More generally, dimensionality reduction techniques like MDS (Kruskal, 1964) might be useful to create positional spaces from the pairwise shortest-path distance matrix or more general functions of the adjacency matrix (Reid et al., 2023).

Dataset Scale

Scale might be the crucial divide between the very successful application of RoPE to natural language as opposed to graphs as studied in this thesis. Point cloud data might

be a good candidate for this as the implied manifold structure is low-dimensional and thus most likely amenable to RoPE on a spectrally induced positional space.

5.4 Summary

We closed with a summary of the main contributions and a discussion of potential follow-up work. Overall, we remain optimistic that future work will be able to build upon the work presented in this thesis. The investigation of Rotary Positional Encodings for graphs will be continued following the roadmap outlined in this chapter.

References

- Abbe, E. (2017). Community detection and stochastic block models. *arXiv preprint arXiv:1703.10146*.
- Alon, U. and Yahav, E. (2020). On the bottleneck of graph neural networks and its practical implications. *arXiv preprint arXiv:2006.05205*.
- Andres, S., Deuschel, J.-D., and Slowik, M. (2016). Heat kernel estimates for random walks with degenerate weights. *Electronic Journal of Probability*, 21(none).
- Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., Muckley, M. J., Rizvi, A., Roberts, C., Sinha, K., Zholus, A., Arnaud, S., Gejji, A., Martin, A., Hogan, F. R., Dugas, D., Bojanowski, P., Khalidov, V., Labatut, P., Massa, F., Szafraniec, M., Krishnakumar, K., Li, Y., Ma, X., Chandar, S., Meier, F., LeCun, Y., Rabbat, M., and Ballas, N. (2025). V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *CoRR*, abs/2506.09985.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Barbero, F., Vitvitskyi, A., Perivolaropoulos, C., Pascanu, R., and Veličković, P. (2024). Round and round we go! what makes rotary positional encodings useful?
- Beaini, D., Passaro, S., Létourneau, V., Hamilton, W., Corso, G., and Liò, P. (2021). Directional graph networks. In *International Conference on Machine Learning*, pages 748–758. PMLR.
- Biyikoglu, T., Leydold, J., and Stadler, P. (2007). *Laplacian Eigenvectors of Graphs: Perron-Frobenius and Faber-Krahn Type Theorems*. Lecture Notes in Mathematics. Springer Berlin Heidelberg.
- Black, M., Wan, Z., Mishne, G., Nayyeri, A., and Wang, Y. (2024). Comparing graph transformers via positional encodings. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org.
- Bo, D., Shi, C., Wang, L., and Liao, R. (2023). Specformer: Spectral graph neural networks meet transformers. *arXiv preprint arXiv:2303.01028*.
- Bochner, S. (1933). Monotone funktionen, stieltjessche integrale und harmonische analyse. *Mathematische Annalen*, 108(1):378–410.
- Bresson, X. and Laurent, T. (2017). Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*.

- Bronstein, M. M., Bruna, J., Cohen, T., and Velicković, P. (2021). Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges. *arXiv preprint arXiv:2104.13478*.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Chen, S., Wong, S., Chen, L., and Tian, Y. (2023). Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*.
- Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., et al. (2020). Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*.
- Choromanski, K., Lin, H., Chen, H., Zhang, T., Sehanobish, A., Likhoshesterov, V., Parker-Holder, J., Sarlos, T., Weller, A., and Weingarten, T. (2022). From block-toeplitz matrices to differential equations on graphs: towards a general theory for scalable masked transformers. In *International Conference on Machine Learning*, pages 3962–3983. PMLR.
- Chung, F. R. K. (1997). *Spectral Graph Theory*. American Mathematical Society, Providence, RI.
- Comanici, G., Bieber, E., Schaeckermann, M., Pasupat, I., Sachdeva, N., Dhillon, I., Blistein, M., Ram, O., Zhang, D., Rosen, E., et al. (2025). Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.
- Dao, T. (2023). Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*.
- Deng, S., Ling, S., and Strohmer, T. (2021). Strong consistency, graph laplacians, and the stochastic block model. *J. Mach. Learn. Res.*, 22(1).
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.
- Dwivedi, V. P. and Bresson, X. (2020). A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699*.
- Dwivedi, V. P., Joshi, C. K., Luu, A. T., Laurent, T., Bengio, Y., and Bresson, X. (2023). Benchmarking graph neural networks. *Journal of Machine Learning Research*, 24(43):1–48.
- Dwivedi, V. P., Luu, A. T., Laurent, T., Bengio, Y., and Bresson, X. (2022a). Graph neural networks with learnable structural and positional representations.

- Dwivedi, V. P., Rampásek, L., Galkin, M., Parviz, A., Wolf, G., Luu, A. T., and Beaini, D. (2022b). Long range graph benchmark. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Erdős, P. and Rényi, A. (1959). On random graphs. *Publicationes Mathematicae*, 6:290–297.
- Fey, M. and Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- Fiedler, M. (1973). Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305.
- Gao, M., Zhang, D., Chen, Y., Zhang, Y., Wang, Z., Wang, X., Li, S., Guo, Y., Webb, G. I., Nguyen, A. T., May, L., and Song, J. (2024). Graphormerdti: A graph transformer-based approach for drug-target interaction prediction. *Computers in Biology and Medicine*, 173:108339.
- Garg, A. (2024). Graph transformers without positional encodings. *arXiv preprint arXiv:2401.17791*.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256. JMLR Workshop and Conference Proceedings.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. (2024). The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Grötschla, F., Xie, J., and Wattenhofer, R. (2024). Benchmarking positional encodings for gnns and graph transformers. *arXiv preprint arXiv:2411.12732*.
- Hamilton, W. L. (2020). Graph representation learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 14(3):1–159.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778.
- He, X., Hooi, B., Laurent, T., Perold, A., LeCun, Y., and Bresson, X. (2023). A generalization of vit/mlp-mixer to graphs. In *International conference on machine learning*, pages 12724–12745. PMLR.
- Hebert, L., Golab, L., and Cohen, R. (2022). Predicting hateful discussions on reddit using graph transformer networks and communal context. In *2022 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT)*, pages 9–17. IEEE.
- Heo, B., Park, S., Han, D., and Yun, S. (2024). Rotary position embedding for vision transformer. In *European Conference on Computer Vision*, pages 289–305. Springer.

- Horn, R. A. and Johnson, C. R. (1985). *Matrix Analysis*. Cambridge University Press.
- Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., and Leskovec, J. (2020). Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33:22118–22133.
- Hua, E., Jiang, C., Lv, X., Zhang, K., Sun, Y., Fan, Y., Zhu, X., Qi, B., Ding, N., and Zhou, B. (2024). Fourier position embedding: Enhancing attention’s periodic extension for length generalization. *arXiv preprint arXiv:2412.17739*.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr.
- Job, S., Tao, X., Cai, T., Li, L., Xie, H., and Yong, J. (2024). Towards causal classification: A comprehensive study on graph neural networks. *arXiv preprint arXiv:2401.15444*.
- Kac, M. (1966). Can one hear the shape of a drum? *The American Mathematical Monthly*, 73(4):1–23.
- Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. (2020). Transformers are rnns: fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning, ICML’20*. JMLR.org.
- Kazemnejad, A., Padhi, I., Natesan Ramamurthy, K., Das, P., and Reddy, S. (2023). The impact of positional encoding on length generalization in transformers. *Advances in Neural Information Processing Systems*, 36:24892–24928.
- Kingma, D. P. and Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Kipf, T. N. and Welling, M. (2016). Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*.
- Knuth, D. E. (1976). *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 2nd edition.
- Kreuzer, D., Beaini, D., Hamilton, W., Létourneau, V., and Tossou, P. (2021). Re-thinking graph transformers with spectral attention. *Advances in Neural Information Processing Systems*, 34:21618–21629.
- Kruskal, J. B. (1964). Nonmetric multidimensional scaling: A numerical method. *Psychometrika*, 29(2):115–129.
- Lanczos, C. (1950). An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *Journal of Research of the National Bureau of Standards*, 45:255–282.
- Lei, J. and Rinaldo, A. (2015). Consistency of spectral clustering in stochastic block models.

- Li, H., Qin, Y., Ou, B., Xu, L., and Xu, R. (2025). Hope: Hybrid of position embedding for length generalization in vision-language models. *arXiv preprint arXiv:2505.20444*.
- Lim, D., Li, X., Hohne, F., and Lim, S.-N. (2021). New benchmarks for learning on non-homophilous graphs. *arXiv preprint arXiv:2104.01404*.
- Lim, D., Robinson, J., Zhao, L., Smidt, T., Sra, S., Maron, H., and Jegelka, S. (2022). Sign and basis invariant networks for spectral graph representation learning. *arXiv preprint arXiv:2202.13013*.
- Lim, K. S. (2024). Benchmarking graph transformers toward scalability for large graphs. Master of engineering in computer science and molecular biology, Massachusetts Institute of Technology.
- Liu, J., Yang, C., Lu, Z., Chen, J., Li, Y., Zhang, M., Bai, T., Fang, Y., Sun, L., Yu, P. S., and Shi, C. (2025). Graph foundation models: Concepts, opportunities and challenges. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(6):5023–5044.
- Liu, X., Yan, H., Zhang, S., An, C., Qiu, X., and Lin, D. (2023). Scaling laws of rope-based extrapolation. *arXiv preprint arXiv:2310.05209*.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022.
- Maron, H., Ben-Hamu, H., Shamir, N., and Lipman, Y. (2019). Invariant and equivariant graph networks. In *International Conference on Learning Representations*.
- Mialon, G., Chen, D., Selosse, M., and Mairal, J. (2021). Graphit: Encoding graph structure in transformers. *arXiv preprint arXiv:2106.05667*.
- Min, E., Chen, R., Bian, Y., Xu, T., Zhao, K., Huang, W., Zhao, P., Huang, J., Ananiadou, S., and Rong, Y. (2022a). Transformer for graphs: An overview from architecture perspective. *arXiv preprint arXiv:2202.08455*.
- Min, E., Rong, Y., Xu, T., Bian, Y., Luo, D., Lin, K., Huang, J., Ananiadou, S., and Zhao, P. (2022b). Neighbour interaction based click-through rate prediction via graph-masked transformer. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*, pages 353–362.
- Morris, C., Ritzert, M., Fey, M., Hamilton, W. L., Lenssen, J. E., Rattan, G., and Grohe, M. (2019). Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 4602–4609.
- Nikolentzos, G. and Vazirgiannis, M. (2020). Random walk graph neural networks. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H., editors, *Advances in Neural Information Processing Systems*, volume 33, pages 16211–16222. Curran Associates, Inc.

- Oono, K. and Suzuki, T. (2019). Graph neural networks exponentially lose expressive power for node classification. *arXiv preprint arXiv:1905.10947*.
- Ostmeier, S., Axelrod, B., Moseley, M. E., Chaudhari, A., and Langlotz, C. (2024). Liere: Generalizing rotary position encodings. *arXiv preprint arXiv:2406.10322*.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32.
- Peebles, W. and Xie, S. (2023). Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205.
- Pei, H., Wei, B., Chang, K. C.-C., Lei, Y., and Yang, B. (2020). Geom-gcn: Geometric graph convolutional networks. *arXiv preprint arXiv:2002.05287*.
- Perez, L. and Wang, J. (2017). The effectiveness of data augmentation in image classification using deep learning. *arXiv preprint arXiv:1712.04621*.
- Platonov, O., Kuznedelev, D., Diskin, M., Babenko, A., and Prokhorenkova, L. (2023). A critical look at the evaluation of gnns under heterophily: Are we really making progress? *arXiv preprint arXiv:2302.11640*.
- Press, O., Smith, N. A., and Lewis, M. (2021). Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*.
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*, pages 28492–28518. PMLR.
- Rampášek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., and Beaini, D. (2022). Recipe for a General, Powerful, Scalable Graph Transformer. *Advances in Neural Information Processing Systems*, 35.
- Reid, I., Choromanski, K., Berger, E., and Weller, A. (2023). General graph random features. *arXiv preprint arXiv:2310.04859*.
- Roy, O. and Vetterli, M. (2007). The effective rank: A measure of effective dimensionality. In *2007 15th European Signal Processing Conference*, pages 606–610.
- Rozemberczki, B., Allen, C., and Sarkar, R. (2021). Multi-scale attributed node embedding. *Journal of Complex Networks*, 9(2):cnab014.
- Saxe, A. M., McClelland, J. L., and Ganguli, S. (2013). Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv preprint arXiv:1312.6120*.
- Schenck, C., Reid, I., Jacob, M. G., Bewley, A., Ainslie, J., Rendleman, D., Jain, D., Sharma, M., Dubey, A., Wahid, A., Singh, S., Wagner, R., Ding, T., Fu, C., Byravan, A., Varley, J., Gritsenko, A., Minderer, M., Kalashnikov, D., Tompson, J., Sindhvani, V., and Choromanski, K. (2025). Learning the ropes: Better 2d and 3d position encodings with string.

- Sen, P., Namata, G., Bilgic, M., Getoor, L., Galligher, B., and Eliassi-Rad, T. (2008). Collective classification in network data. *AI Magazine*, 29(3):93.
- Shirzad, H., Vellingker, A., Venkatachalam, B., Sutherland, D. J., and Sinop, A. K. (2023). Expformer: Sparse transformers for graphs. In *International Conference on Machine Learning*, pages 31613–31632. PMLR.
- Spielman, D. A. (2009). Spectral graph theory, fall 2009: Lecture 2 - laplacians and adjacency matrices. Lecture Notes, Applied Mathematics 561/ Computer Science 662, Yale University. Accessed: 2024.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. (2023). Roformer: Enhanced transformer with rotary position embedding.
- Tang, J., Sun, J., Wang, C., and Yang, Z. (2009). Social influence analysis in large-scale networks. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '09, page 807–816, New York, NY, USA. Association for Computing Machinery.
- Tang, J., Wang, Z., Chen, S., Zhou, S., Chen, J., and Bu, J. (2025). OpenGT: A comprehensive benchmark for graph transformers. *arXiv preprint arXiv:2506.04765*.
- Tönshoff, J., Ritzert, M., Rosenbluth, E., and Grohe, M. (2023). Where did the gap go? reassessing the long-range graph benchmark. *arXiv preprint arXiv:2309.00367*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph Attention Networks. In *ICLR*.
- Vellingker, A., Sinop, A. K., Ktena, I., Veličković, P., and Gollapudi, S. (2022). Affinity-aware graph networks.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., and SciPy 1.0 Contributors (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272.
- Von Luxburg, U. (2007). A tutorial on spectral clustering. *Statistics and computing*, 17(4):395–416.
- Vulpus, L. and Höfs, C. (2025). Rotational positional encodings for graphs. Project Report L65: Geometric Deep Learning, University of Cambridge.

- Wang, S., Huang, J., Chen, Z., Song, Y., Tang, W., Mao, H., Fan, W., Liu, H., Liu, X., Yin, D., et al. (2024). Graph machine learning in the era of large language models (llms). *ACM Transactions on Intelligent Systems and Technology*.
- Yanardag, P. and Vishwanathan, S. (2015). Deep graph kernels. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '15, page 1365–1374, New York, NY, USA. Association for Computing Machinery.
- Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., and Liu, T.-Y. (2021). Do transformers really perform badly for graph representation? *Advances in neural information processing systems*, 34:28877–28888.
- You, J., Ying, R., and Leskovec, J. (2020). Design space for graph neural networks. In *NeurIPS*.
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., and Smola, A. (2018). Deep sets.

Appendix A

Notation

This chapter establishes the general formatting conventions and mathematical notation used throughout this thesis.

Discrete Variables Since complex numbers are not used in this work, i exclusively denotes a scalar index, most commonly that of a node in a graph or a token in a sequence, and never the imaginary unit. For $n \in \mathbb{N}$, we write $[n] := \{0, 1, \dots, n-1\}$.

Matrices and Vectors: Matrices are denoted in bold uppercase letters (e.g., $\mathbf{A}$, $\mathbf{L}$), while vectors are denoted in bold lowercase letters (e.g., $\boldsymbol{\phi}$, $\boldsymbol{\psi}$). The identity matrix is denoted as $\mathbf{I}$. We use the notation $\mathbf{1}_N$ for the all-ones vector of length N , which is used to succinctly write summations or repetitions in matrix form.

Formulae containing braces or comma-separated variables are to be read as summarising repeated equations instantiated for each variable in the braces.

Concatenation or stacking of matrices along dimension d is denoted as $[\dots]_d$.

Wherever it is clear from context, scalar functions may be applied element-wise to tensors, often only along the last dimension.

Operators and Functions: Special functions and transformations are denoted with calligraphic letters (e.g., $\mathcal{R}$ for the rotary transformation) or descriptive text (e.g., softmax and diag). The Heaviside step function is denoted as:

$$\theta(x) := \begin{cases} 1 & \text{if } x > 0 \\ \frac{1}{2} & \text{if } x = 0 \\ 0 & \text{otherwise} \end{cases}$$

The sign function is denoted as:

$$\text{sgn}(x) := \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$$

The symbol $\mathcal{R}$ denotes the stacking of a vector of angles into a block-diagonal rotation matrix (cf. [Equation \(2.4\)](#)).

The softmax function denotes the following normalisation function, which is most often applied to the rows of a matrix:

$$\text{softmax}(\mathbf{x})_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

Graph Notation: Graphs are denoted as $G = (V, E)$, where $V \subseteq \mathbb{N}$ is the vertex set and $E \subseteq V \times V$ is the edge set. The number of vertices is denoted as $N = |V|$.

Complexity Notation: Computational complexity is expressed using big O notation (e.g., $\Theta(N^3)$ for cubic complexity) ([Knuth, 1976](#)).

Appendix B

Proofs

B.1 Recovering RoPE

Consider the DeepSet encoder [Equation \(2.9\)](#) for $k = 2$ with no f_{post} and the eigenvalues and eigenvectors from [Equation \(2.8\)](#)

$$\begin{aligned}\lambda_0 &= 0 & \psi_0(i) &= 1 \\ \lambda_1 &= 2 - 2 \cos \frac{\pi}{N} & \psi_1(i) &= \cos \left(\frac{\pi}{N} \left(i + \frac{1}{2} \right) \right)\end{aligned}$$

and considering only the first output dimension of f_{enc} we want

$$t_i \stackrel{!}{=} \pm i = f(\lambda_0, \pm \psi_0) + f(\lambda_1, \pm \psi_1)$$

This constraint can be solved for $f(\lambda, \psi)$. From the eigenvalue, we have direct access to the number of nodes N :

$$N(\lambda) = \frac{\pi}{\arccos \left(1 - \frac{\lambda}{2} \right)}$$

The evident singularity at $\lambda = 0$ has to be cut out artificially. This is possible as N is bounded in practice so despite $\lambda_1(N) \rightarrow 0$ as $N \rightarrow \infty$ we have $\lambda_1(N) > 0$. Thus,

we conveniently define $f(0, \pm 1) = 0$ to obtain

$$\begin{aligned} f(\lambda, \psi) &= \operatorname{sgn}(\psi) \left(\frac{N(\lambda)}{\pi} \arccos |\psi| - \frac{1}{2} \right) + \theta(-\psi)(N(\lambda) - 1) \\ &= \operatorname{sgn}(\psi) \left(\frac{\arccos |\psi|}{\arccos \left(1 - \frac{\lambda}{2}\right)} - \frac{1}{2} \right) + \theta(-\psi)(N(\lambda) - 1) \end{aligned}$$

and finally

$$t_i = f(0, \pm 1) + f(\lambda_1, \pm \psi_1(i)) = \pm i$$